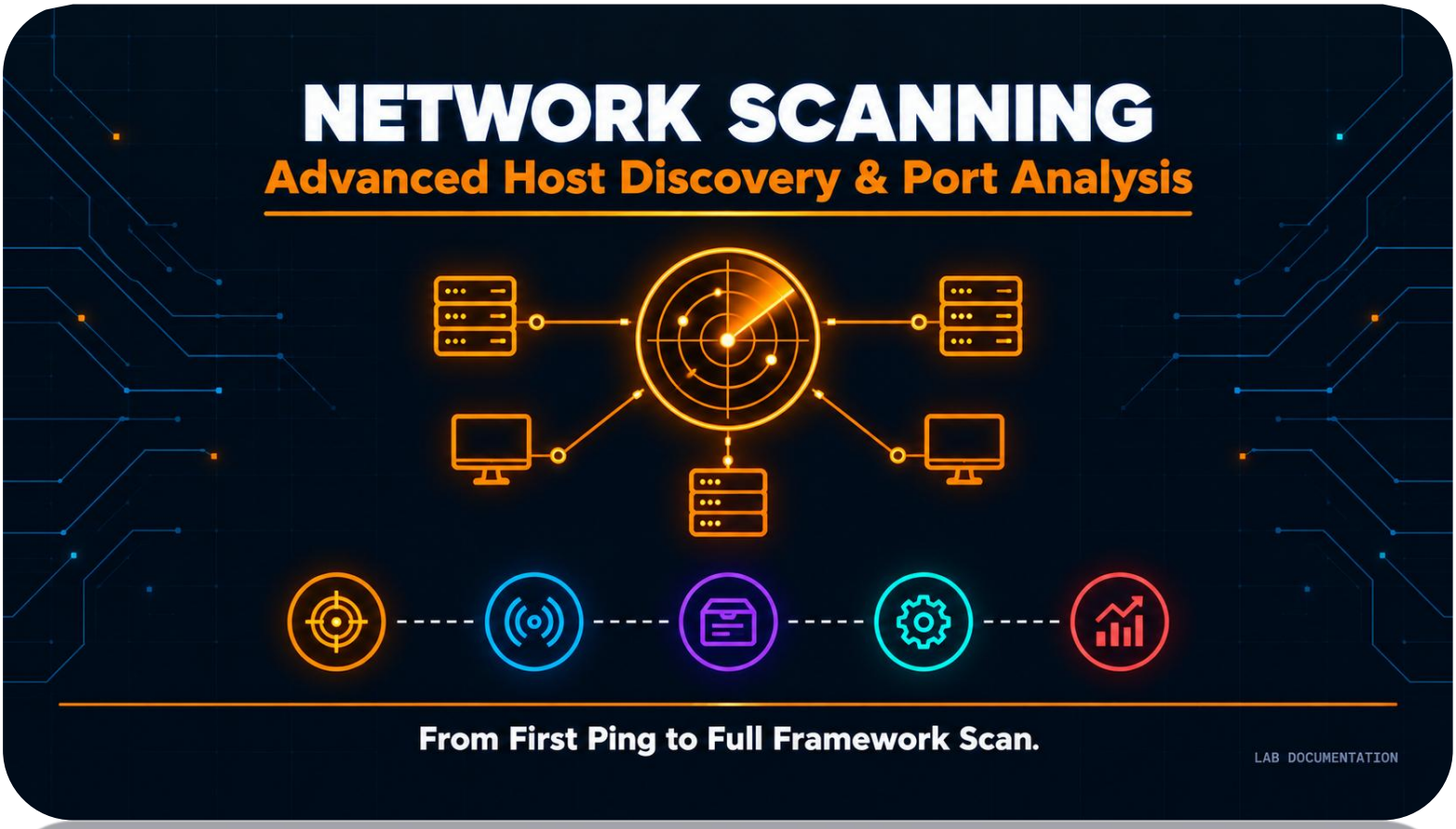


Prepared by: ***Abdul Wahid***
Document Type: ***Lab Report***
Category: ***Network Scanning & Reconnaissance***

Contents

Installation Summary 7

Metasploitable 2 (Linux Target) 7

Ubuntu + DVWA (Web Application Target) 9

Windows 10 (Client Target) 11

Advanced Host Discovery 12

Custom Packet Crafting with hping3 14

TCP SYN Discovery 14

TCP SYN-ACK Probe 14

TCP FIN Scan 15

TCP RST Probe 15

TCP NULL Scan 15

Zombie / Idle Scanning 16

Zombie Host Identification 16

Idle Scan — Metasploitable via Windows Zombie 17

Idle Scan — DVWA via Windows Zombie 17

Firewall Behavior Analysis During Discovery 18

ICMP Filtering Detection 18

TCP Port Filtering Detection 19

Response Code Interpretation (Multi-Port Sweep) 19

Timing & Rate Control 20

Rate-Limited Discovery 20

Parallel Processing Optimization 21

Response Analysis & Network Topology Mapping 22

TTL Analysis for Network Distance 22

Response Pattern Analysis 23

Custom Discovery Script Development 24

Multi-Technique Discovery Script (TCP probes via hping3) 24

Firewall Analysis — DVWA (192.168.100.142) 24

Port Scanning Analysis 26

Port State Classification Mastery 27
SYN Stealth Scan (Baseline) 27
TCP Connect Scan (Verification) 28
ACK Scan (Firewall Testing) 29
Target-Specific Scanning Analysis 30
Metasploitable (192.168.100.148) — Linux 30
DVWA (192.168.100.142) Web Server 34
Firewall Pattern Detection 36
Automated Response Categorization 36
Adaptive Scanning Strategies 37
Target Responsiveness Assessment Script 37
Adaptive Scan Execution 38
Statistical Analysis 39
Scanning Automation & Optimization 41
Advanced Evasion During Scanning 42
Fragmented Port Scanning 42
Source Port Manipulation 45
Decoy Scanning 47
Combined Evasion 49
Custom Scanning Automation 50
Adaptive Multi-Target Scanner 50
Port Scanning Analysis Using Zenmap (GUI-Based) 53
Launch Zenmap and Confirm Reachability 54
TCP Connect Scan (-sT) 55
TCP SYN / Stealth Scan (-sS) 56
Xmas Scan (-sX) 57
TCP Maimon Scan (-sM) 58
ACK Flag Probe Scan (-sA) 58
Network Scanning Using Metasploit 60
Nmap from Inside Metasploit 61
SYN Scan Module (auxiliary/scanner/portscan/syn) 64

TCP Connect Scan Module (auxiliary/scanner/portscan/tcp) 65
SMB Version Scan (auxiliary/scanner/smb/smb_version) 66
Conclusion 68

Executive Summary

This document describes the setup and execution of an isolated network scanning lab, designed to safely practice advanced offensive security scanning techniques. The environment is fully contained within a private, internet-free virtual network, enabling unrestricted, legally and ethically sound practice of host discovery, port scanning, evasion, and reconnaissance methodologies.

Across five structured projects, the lab progresses from foundational host discovery through advanced port analysis, scan automation and evasion, GUI-based tooling, and framework-integrated scanning — building a complete, validated understanding of the scanning phase of a security assessment.

Objectives

- Establish a fully isolated lab environment for safe practice
- Practice advanced network scanning techniques
- Master host discovery, port enumeration, and OS fingerprinting
- Learn reconnaissance and network enumeration methodologies
- Build professional documentation and reporting habits
- Develop hands-on, practical cybersecurity skills

Lab Scope & Network Configuration

Network Configuration

Parameter	Value
Network Type	VirtualBox Host-Only Network
Network Range	192.168.100.0/24
Isolation Level	Complete (no internet access)
Safety Level	Safe for unrestricted practice



Lab Machines

Role	Machine	IP Address
Attacker / Scanner	Kali Linux	DHCP-assigned
Target 1	Metasploitable 2 (Linux)	192.168.100.148
Target 2	DVWA (Web Application, via Docker)	192.168.100.142
Target 3	Windows 10	192.168.100.145

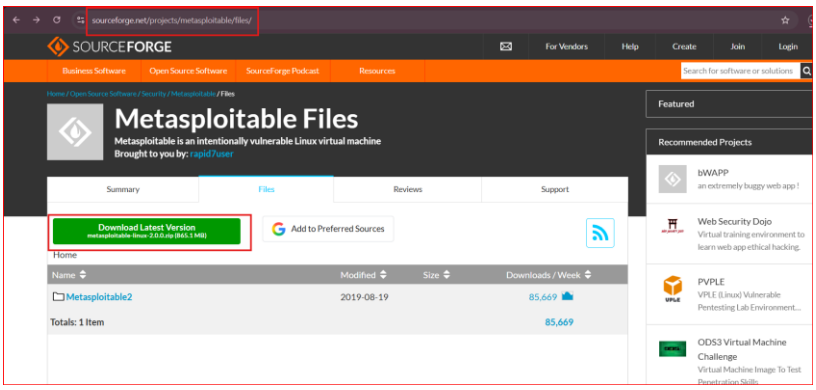
Installation Summary

Metasploitable 2 (Linux Target)

Metasploitable 2 is a deliberately vulnerable Linux distribution used as the primary target for host- and service-level scanning exercises. It was included in this lab to:

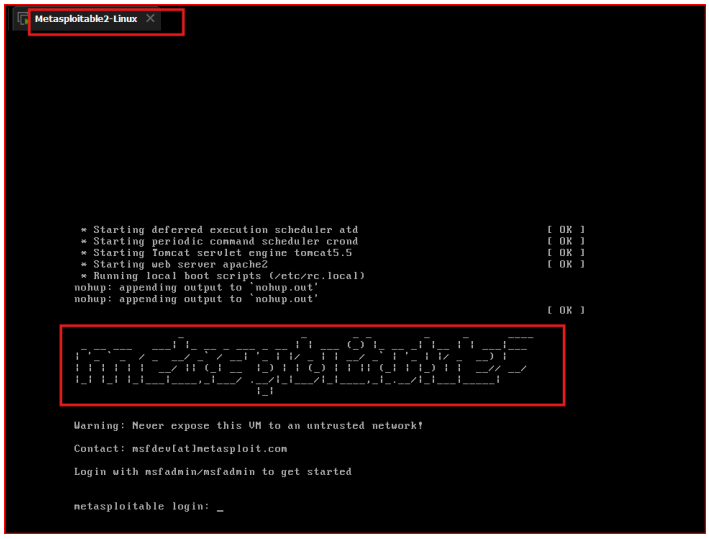
- Provide a Linux-based target running multiple exposed services (FTP, SSH, Telnet, SMTP, RPC, SMB, MySQL, PostgreSQL)
- Enable testing of scan types (SYN, FIN, NULL, ACK) against a realistic, unpatched service profile
- Serve as a stable, well-documented baseline target for validating scan results

Download metasploitable-linux-2.0.0.zip (865 MB) from SourceForge

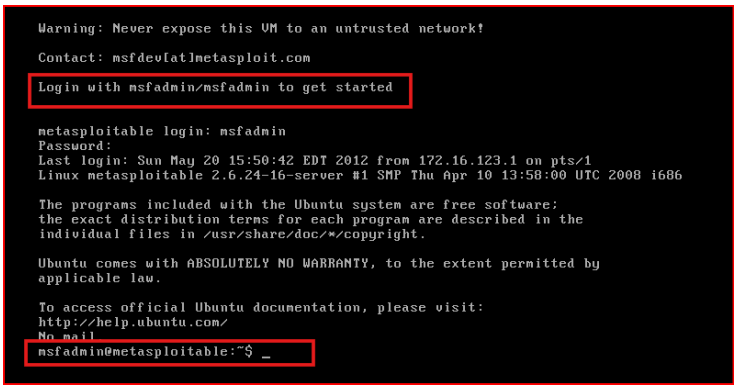


Extract the archive and import the OVA file into VirtualBox/VMware

Start the VM and allow the boot sequence to complete

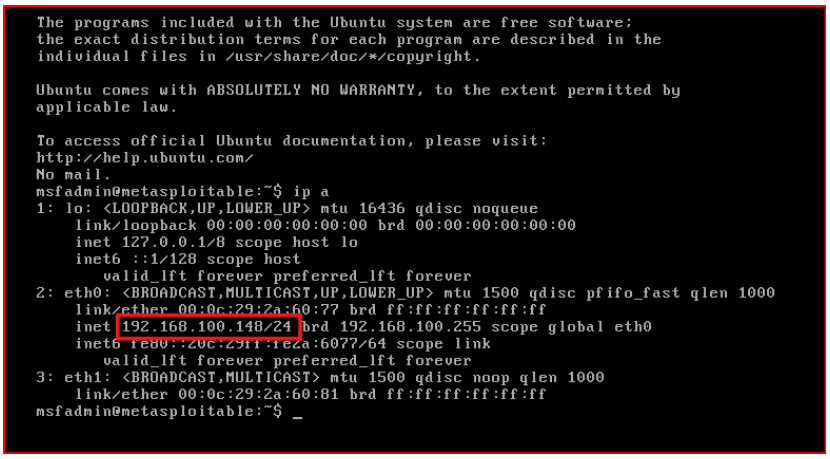


Log in with credentials: msfadmin / msfadmin

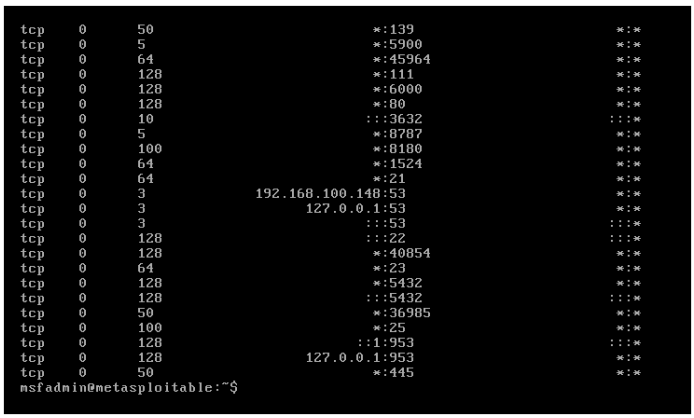


Verify configuration:

`ip a → confirms IP 192.168.100.148`



ss -tuln → confirms active listening services (FTP, SSH, Telnet, SMTP, DNS, HTTP, NetBIOS/SMB, RPC, MySQL, PostgreSQL, VNC, X11, IRC, etc.)

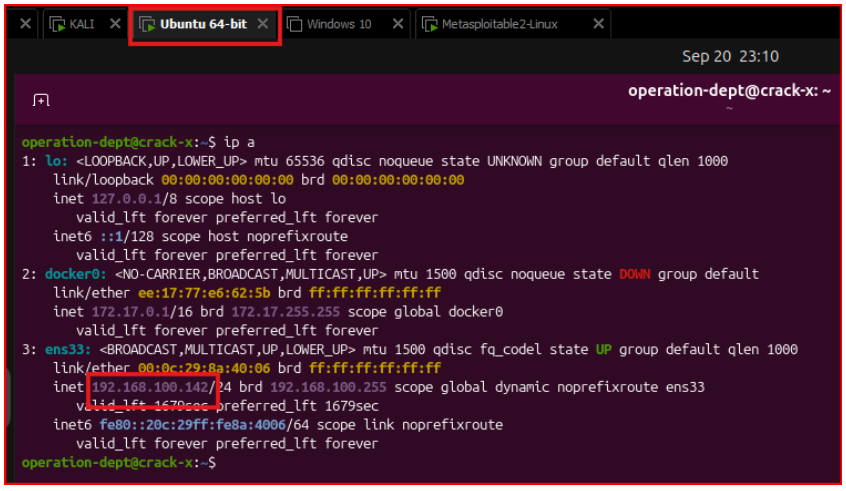


Ubuntu + DVWA (Web Application Target)

DVWA (Damn Vulnerable Web Application) was deployed as a separate web-application target to introduce HTTP/HTTPS-based scanning into the lab. It was included to:

- Represent a web-facing host, distinct from the OS-level services exposed by Metasploitable
- Enable analysis of port and firewall behavior specific to web server ports (80, 443, 8080)
- Provide a foundation for future web application enumeration and assessment exercises

Create a new Ubuntu 64-bit VM



Install Docker:

```
sudo apt-get update && sudo apt-get install -y docker.io
```

Clean any pre-existing containers

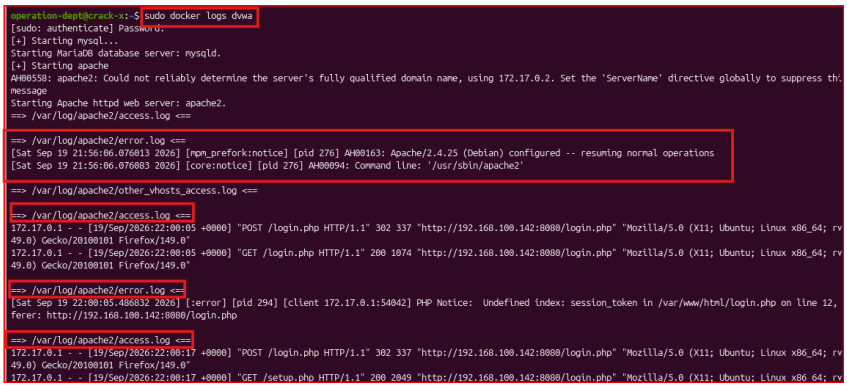
Deploy DVWA:

```
sudo docker run -d -p 8080:80 --name dvwa vulnerables/web-dvwa
```



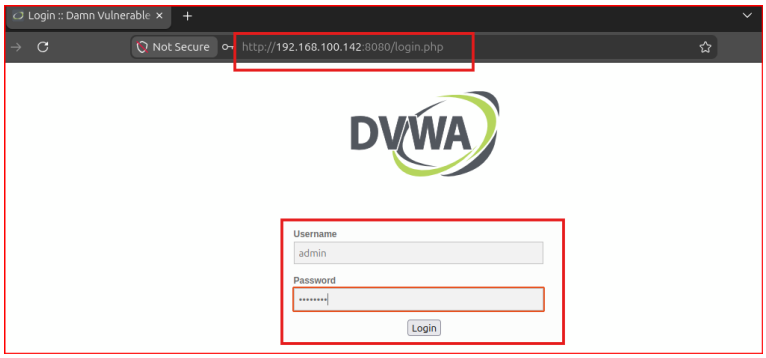
Verify container status:

```
sudo docker logs dvwa
```

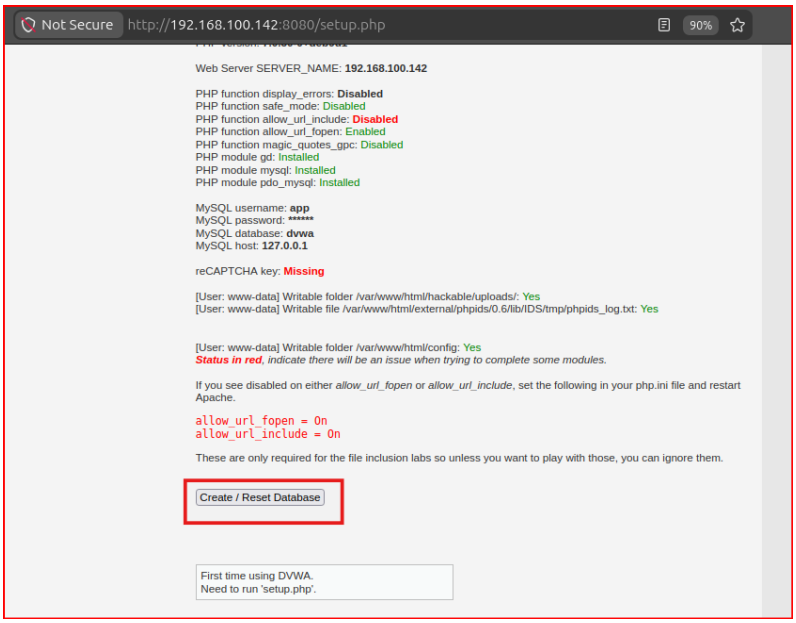


Access via browser: http://192.168.100.142:8080

Login: admin / password

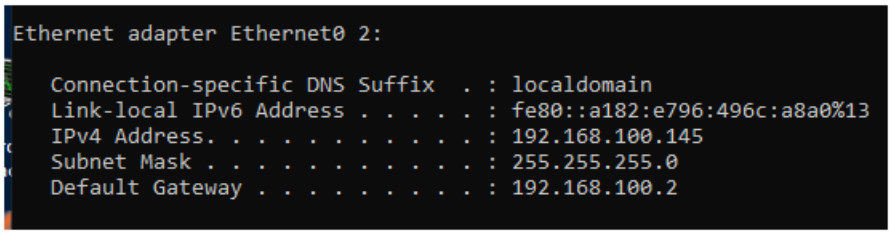


Click Create / Reset Database on first use



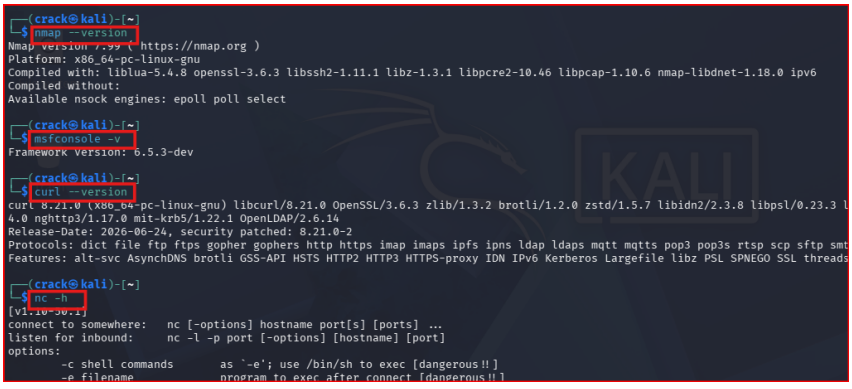
Windows 10 (Client Target)

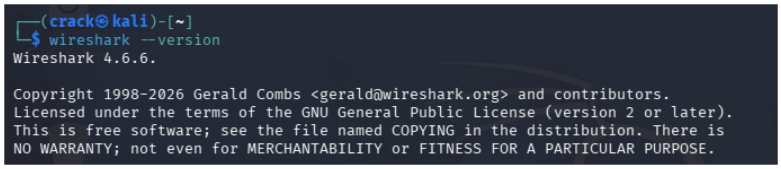
1. Create Windows 10 64-bit VM and complete a standard installation from ISO
2. Configure networking: Control Panel → Network Settings → Ethernet adapter properties → enable DHCP
3. Confirm IP assignment (192.168.100.145) via ipconfig



Tools Verification (Kali Linux)

Tool	Verification Command
Nmap	nmap --version
Metasploit Framework	msfconsole -v
cURL	curl --version
Netcat	nc -h
Wireshark	wireshark --version





Advanced Host Discovery

Objectives

- Master advanced host discovery techniques beyond standard ping sweeps
- Use custom packet crafting for discovery
- Understand firewall behavior during the discovery phase
- Implement low-and-slow discovery techniques
- Perform zombie/idle scanning
- Analyze network responses for topology mapping
- Optimize discovery for large networks
- Develop custom discovery scripts



Focus: Advanced host discovery only (scanning phase) no service/version/OS enumeration performed at this stage.

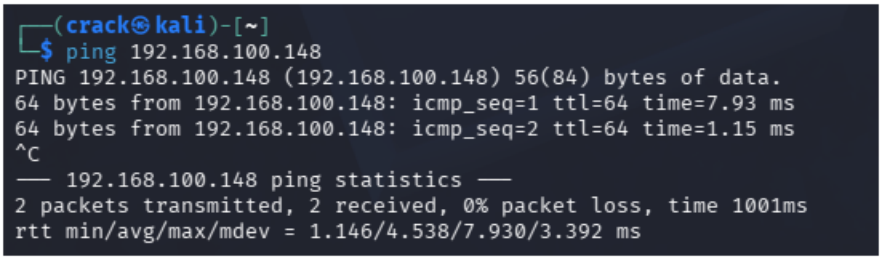
Target	IP Address
Metasploitable	192.168.100.148
DVWA	192.168.100.142
Windows	192.168.100.145

Network: 192.168.100.0/24

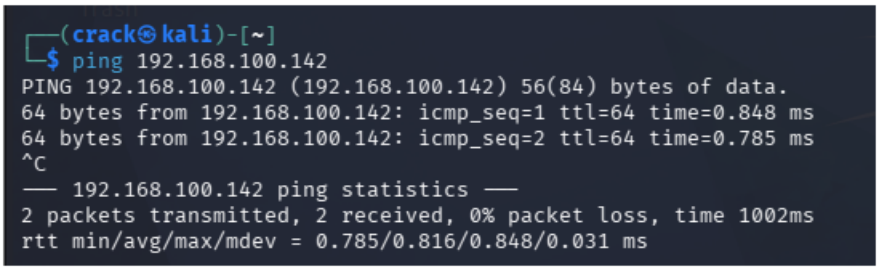
Baseline Network Reachability Check

Before running discovery techniques, basic connectivity was confirmed against each target.

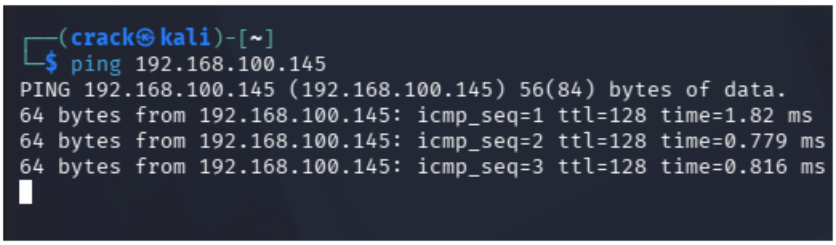
```
ping -c 4 192.168.100.148   # Metasploitable
```




```
ping -c 4 192.168.100.142   # DVWA
```




```
ping -c 4 192.168.100.145   # Windows
```



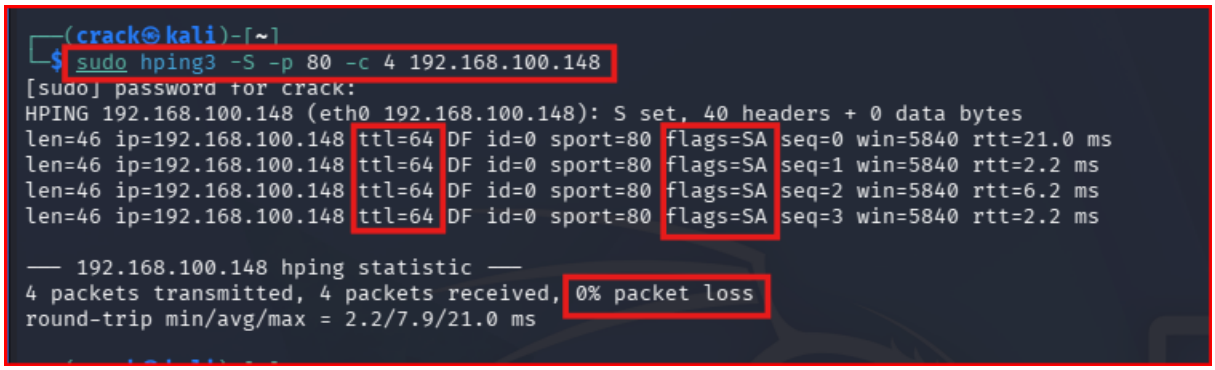
Finding: All three targets responded with 0% packet loss, confirming full network reachability before proceeding to advanced discovery techniques.

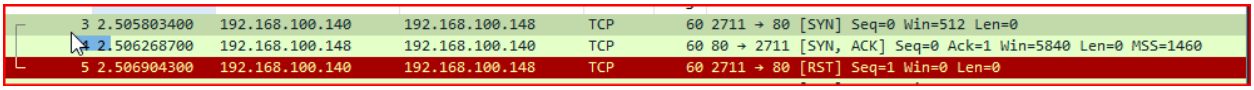
Custom Packet Crafting with hping3

Purpose: Move beyond standard ICMP ping and manually craft TCP packets with specific flag combinations to study how hosts and firewalls respond — the foundation of advanced discovery.

TCP SYN Discovery

```
hping3 -S -p 80 -c 4 192.168.100.148
```

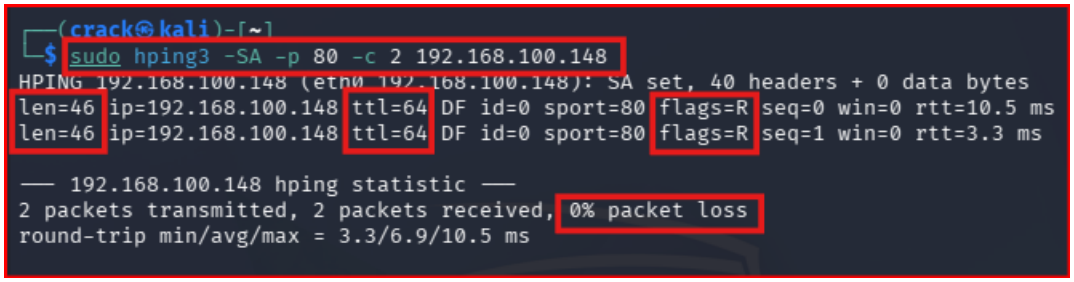


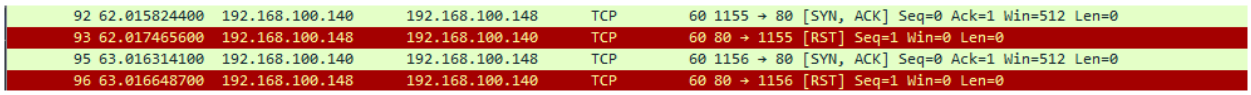


Finding: Target responded with SYN-ACK flags on all 4 probes with 0% packet loss, confirming the host is alive and port 80 is open.

TCP SYN-ACK Probe

```
hping3 -SA -p 80 -c 2 192.168.100.148
```

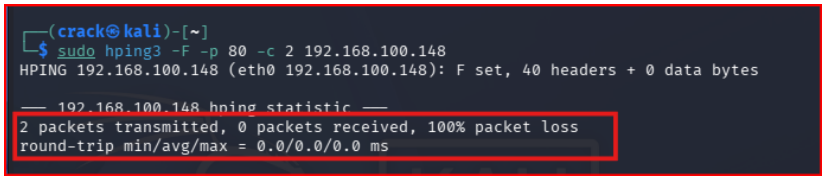


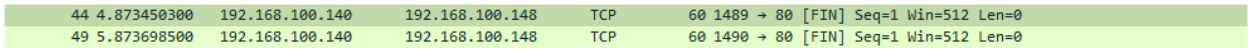


Finding: Target replied with RST flags, indicating the host is alive; an unsolicited SYN-ACK probe is correctly rejected by the target's TCP stack.

TCP FIN Scan

```
hping3 -F -p 80 -c 2 192.168.100.148
```

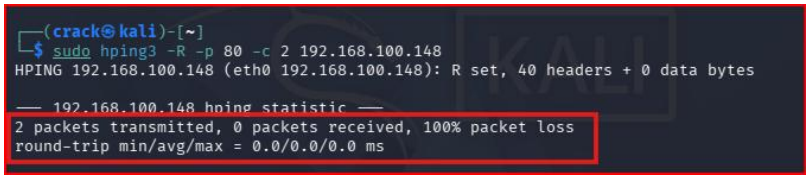


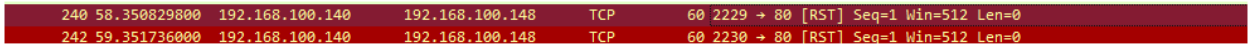


Finding: No response received (100% packet loss). Per FIN scan behavior, this indicates the port is either open or filtered, since only closed ports typically reply with RST.

TCP RST Probe

```
hping3 -R -p 80 -c 2 192.168.100.148
```

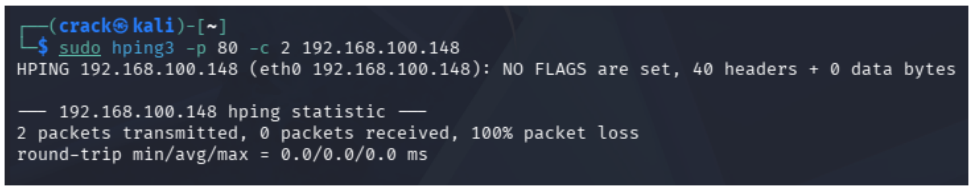


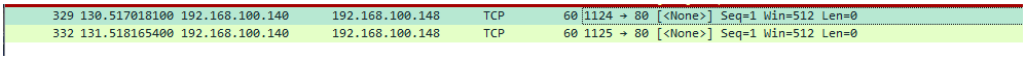


Finding: No response received. Sending an unsolicited RST does not trigger a reply, which is expected TCP behavior and does not indicate host status by itself.

TCP NULL Scan

```
hping3 -p 80 -c 2 192.168.100.148
```





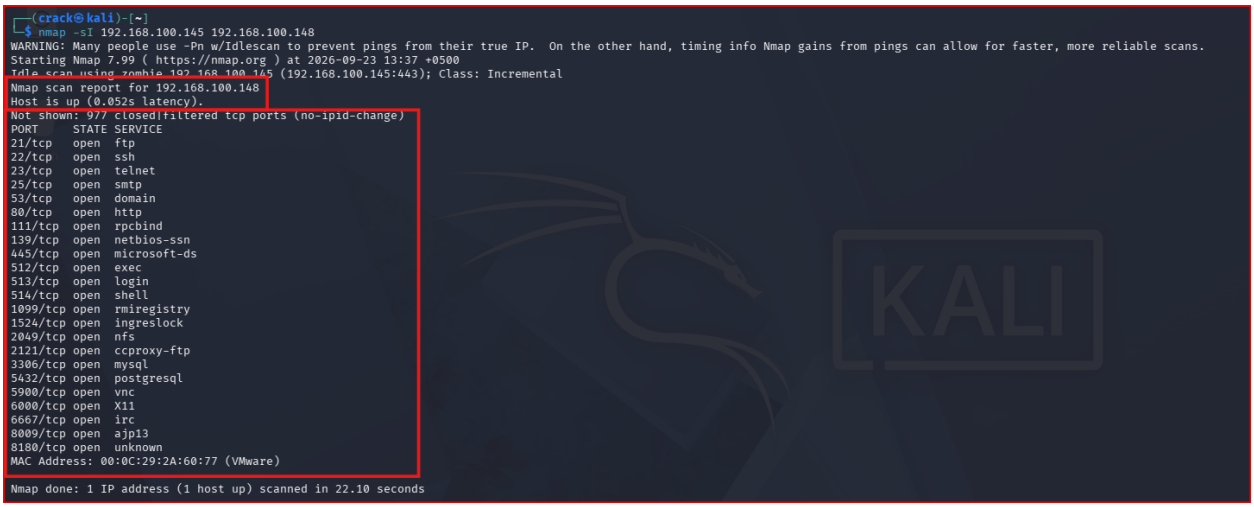
Finding: No response received. Under NULL scan logic, a silent target suggests the port is open or filtered, consistent with the FIN scan result above.

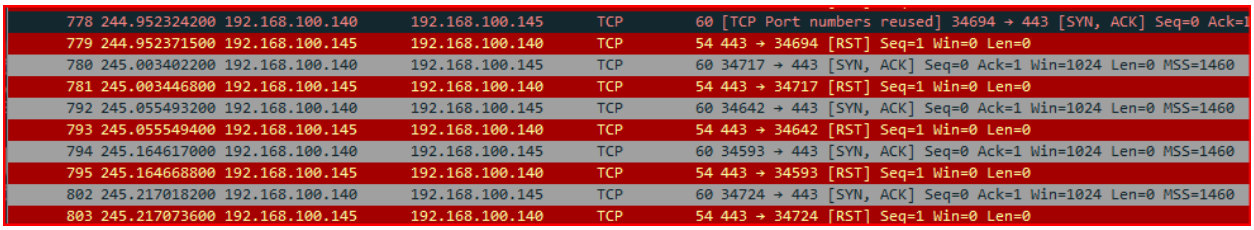
Zombie / Idle Scanning

Purpose: Scan a target through a third-party "zombie" host to avoid revealing the attacker's real IP address — an advanced stealth technique.

Zombie Host Identification

```
nmap -sI 192.168.100.145 192.168.100.148
```

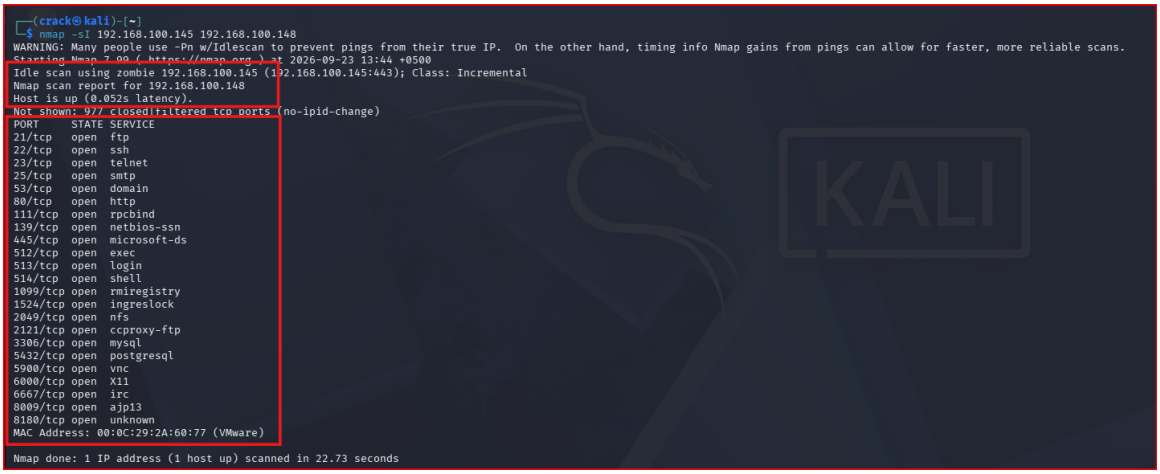


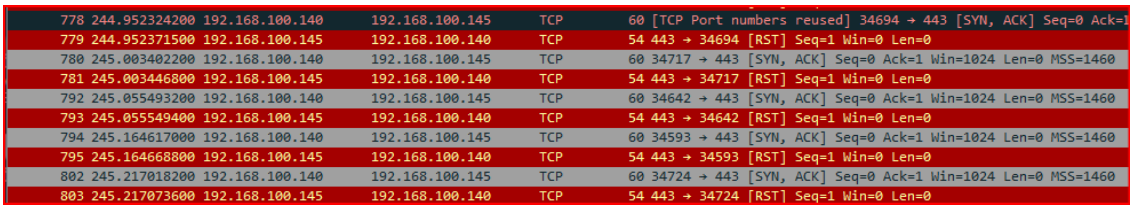


Finding: The Windows host was tested as a candidate zombie due to its predictable IP ID sequence, a prerequisite for idle scanning to work reliably.

Idle Scan — Metasploitable via Windows Zombie

`nmap -sI 192.168.100.145 192.168.100.148`

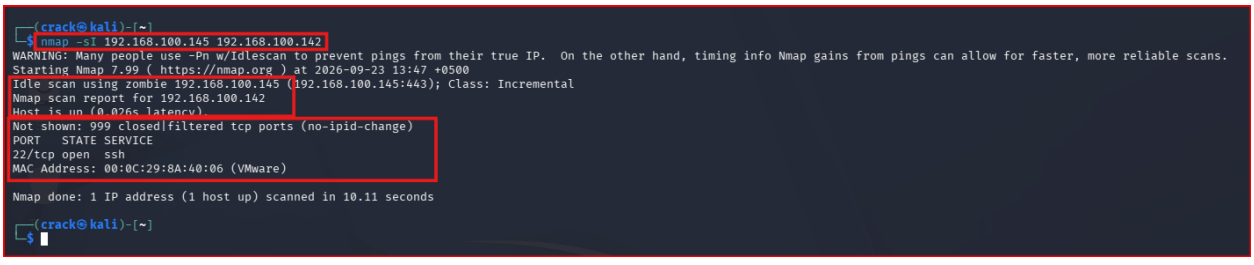




Finding: The scan successfully enumerated open ports on Metasploitable (FTP, SSH, Telnet, SMTP, DNS, HTTP, RPC, SMB, and others) without sending any packets directly from the Kali attacking machine — all probes were reflected through the zombie host.

Idle Scan — DVWA via Windows Zombie

`nmap -sI 192.168.100.145 192.168.100.142`





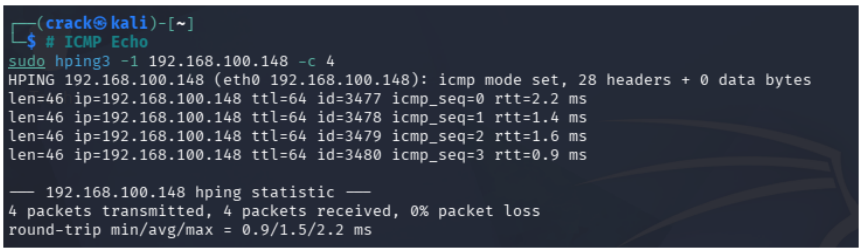
Finding: Only port 22 (SSH) was identified as open on DVWA, confirming the zombie technique also works correctly against a second, differently configured target.

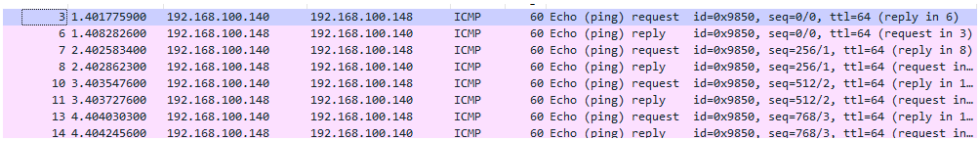
Firewall Behavior Analysis During Discovery

Purpose: Detect and interpret firewall filtering rules by analyzing how targets respond to different probe types.

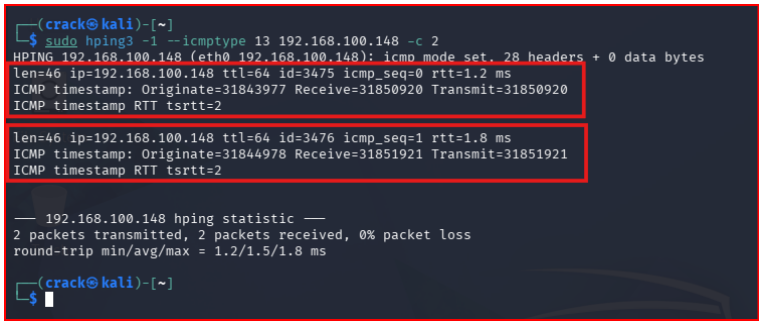
ICMP Filtering Detection

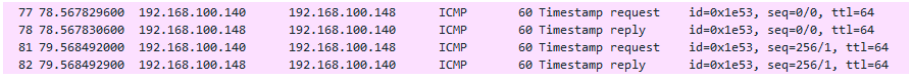
```
sudo hping3 -1 192.168.100.148 -c 4                    # ICMP Echo
```






```
sudo hping3 -1 --icmptype 13 192.168.100.148 -c 2 # ICMP Timestamp
```

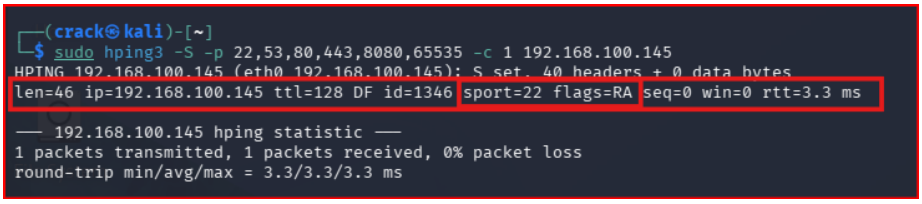


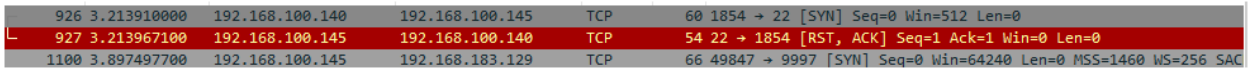


Finding: The target responded to both ICMP Echo and ICMP Timestamp requests, indicating ICMP is not filtered on this host — useful for confirming host liveness through multiple ICMP types.

TCP Port Filtering Detection

hping3 -S -p 22,53,80,443,8080,65535 -c 1 192.168.100.145

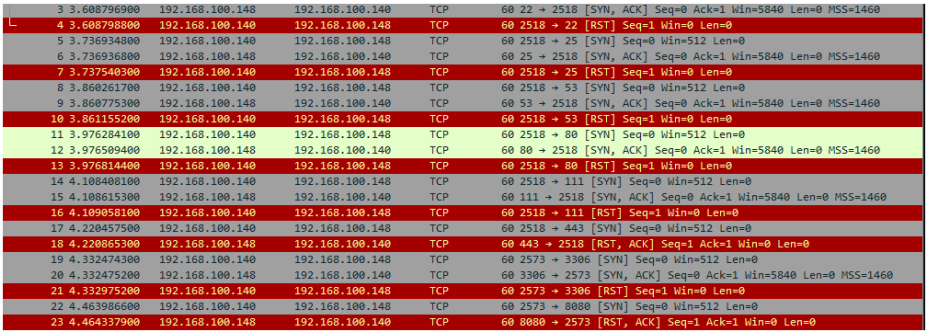




Finding: Response showed flags=RA (RST-ACK) on the probed port, indicating the host is alive but that specific port is closed.

Response Code Interpretation (Multi-Port Sweep)





Finding: Ports 22, 25, 53, 80, 111, and 3306 returned SA (open), while ports 443 and 8080 returned RA (closed) giving a clear per-port open/closed map for Metasploitable.

Response Code Reference

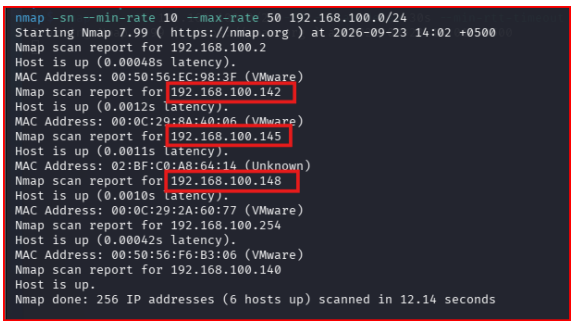
Response	Meaning
flags=SA (SYN-ACK)	Host alive, port **OPEN**
flags=RA (RST-ACK)	Host alive, port **CLOSED**
UNREACHABLE	Host likely behind a firewall
Timeout	Firewall silently dropping packets
No response	Filtered or host down

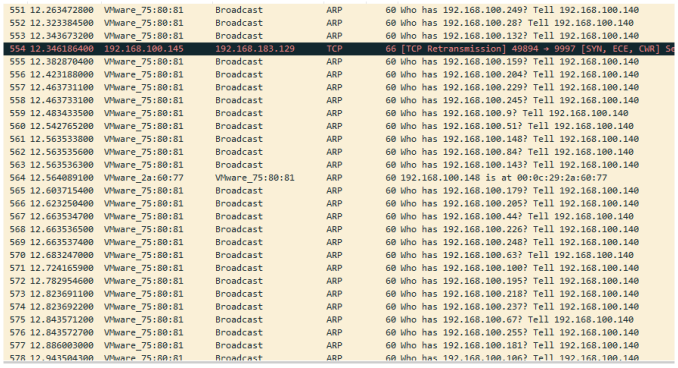
Timing & Rate Control

Purpose: Control discovery speed for stealth (evading detection) or performance (large-scale sweeps).

Rate-Limited Discovery

```
nmap -sn --min-rate 10 --max-rate 50 192.168.100.0/24
```

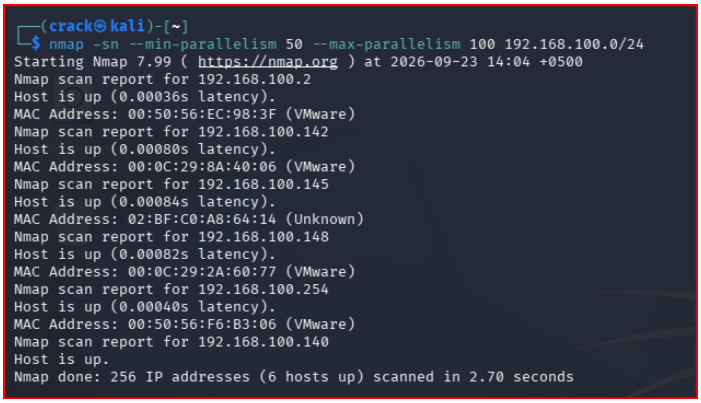


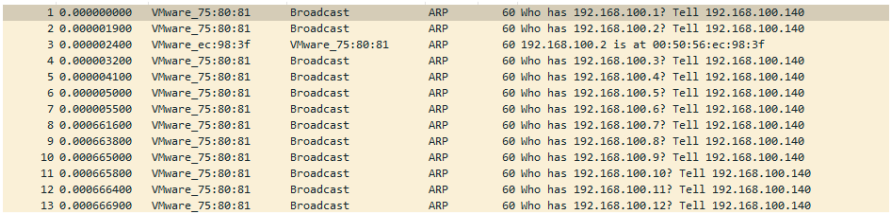


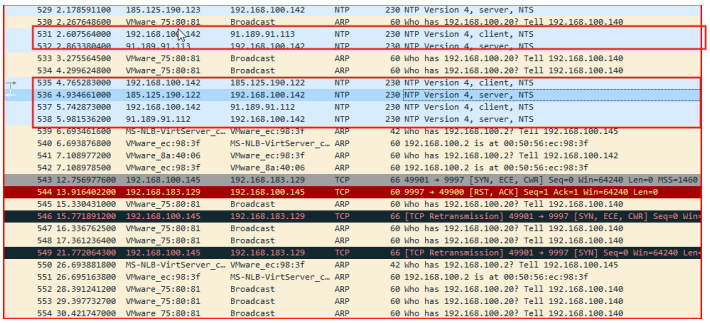
Finding: Scan identified 6 live hosts on the subnet in ~12 seconds while intentionally throttled — a deliberately slower, stealthier sweep.

Parallel Processing Optimization

```
nmap -sn --min-parallelism 50 --max-parallelism 100
192.168.100.0/24
```







Finding: The same 6 hosts were identified in ~2.7 seconds — over 4x faster than the rate-limited scan, demonstrating the trade-off between speed and stealth.

Response Analysis & Network Topology Mapping

Purpose: Infer network topology and distance from response metadata (scanning phase only).

TTL Analysis for Network Distance

```
echo "TTL Analysis for Network Topology Mapping"

for IP in 192.168.100.{140..150}; do
  TTL=$(ping -c 1 -W 1 $IP 2>/dev/null | grep "ttl=" | grep -oP 'ttl=\K[0-9]+')

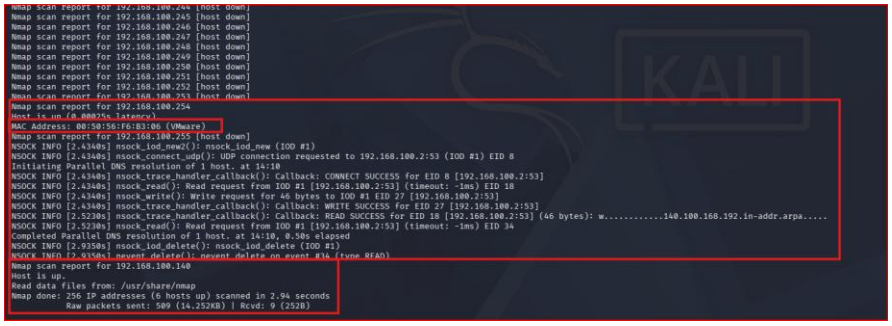
  if [ ! -z "$TTL" ]; then
    HOPS=$((64 - TTL))  # Assuming Linux (starts at 64)
    echo "$IP - TTL: $TTL - Estimated hops: $HOPS"
  fi
done
TTL Analysis for Network Topology Mapping
192.168.100.140 - TTL: 64 - Estimated hops: 0
192.168.100.142 - TTL: 64 - Estimated hops: 0
192.168.100.145 - TTL: 128 - Estimated hops: -64
192.168.100.148 - TTL: 64 - Estimated hops: 0
```

1	0.000000000	192.168.100.145	192.168.183.129	TCP	66	49915 → 9997 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK…
4	1.851572100	192.168.100.140	192.168.100.142	ICMP	98	Echo (ping) request id=0x3bfb, seq=1/256, ttl=64 (reply in 5)
5	1.852167500	192.168.100.142	192.168.100.140	ICMP	98	Echo (ping) reply id=0x3bfb, seq=1/256, ttl=64 (request in…
14	3.881385800	192.168.100.140	192.168.100.145	ICMP	98	Echo (ping) request id=0x3bfe, seq=1/256, ttl=64 (reply in 1…
15	3.881597200	192.168.100.145	192.168.100.140	ICMP	98	Echo (ping) reply id=0x3bfe, seq=1/256, ttl=128 (request i…
20	4.880341900	192.168.100.140	103.55.68.158	NTP	90	NTP Version 4, client
24	4.940712200	103.55.68.158	192.168.100.140	NTP	90	NTP Version 4, server
25	5.913736400	192.168.100.140	192.168.100.148	ICMP	98	Echo (ping) request id=0x3c01, seq=1/256, ttl=64 (reply in 2…
26	5.913738100	192.168.100.148	192.168.100.140	ICMP	98	Echo (ping) reply id=0x3c01, seq=1/256, ttl=64 (request in…

Finding: Linux-based hosts (.140, .142, .148) returned a TTL of 64 (0 estimated hops), while the Windows host (.145) returned TTL 128 — a useful fingerprint for distinguishing OS families purely from response headers, without deeper enumeration.

Response Pattern Analysis

```
nmap -sn -PE --packet-trace 192.168.100.0/24 -v
```



1	0.000000000	MS-NLB-VirtServer_c…	Broadcast	ARP	42	Who has 192.168.100.75? Tell 192.168.100.145
2	0.905797100	MS-NLB-VirtServer_c…	Broadcast	ARP	42	Who has 192.168.100.75? Tell 192.168.100.145
3	1.075441500	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.1? Tell 192.168.100.140
4	1.075443300	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.2? Tell 192.168.100.140
5	1.075444100	VMware_ec:98:3f	VMware_75:80:81	ARP	60	192.168.100.2 is at 00:50:56:ec:98:3f
6	1.075445100	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.3? Tell 192.168.100.140
7	1.075446000	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.4? Tell 192.168.100.140
8	1.075841300	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.5? Tell 192.168.100.140
9	1.075841900	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.6? Tell 192.168.100.140
10	1.076091000	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.7? Tell 192.168.100.140
11	1.076091800	VMware_75:80:81	Broadcast	ARP	60	Who has 192.168.100.8? Tell 192.168.100.140

565	16.665867500	192.168.100.2	192.168.100.145	DNS	229	Standard query response 0x24e8 A v10.events.data.microsoft.c
566	16.665908500	192.168.100.145	192.168.100.2	ICMP	257	Destination unreachable (Port unreachable)
567	16.892113400	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
568	16.892272000	192.168.100.145	20.42.73.26	TCP	54	49922 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
569	16.892949500	192.168.100.145	20.42.73.26	TLSv1.2	266	Client Hello (SNI=v10.events.data.microsoft.com)
570	16.893480900	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=1 Ack=213 Win=64240 Len=0
571	16.905739900	MS-NLB-VirtServer_c…	Broadcast	ARP	42	Who has 192.168.100.75? Tell 192.168.100.145
572	17.214495300	20.42.73.26	192.168.100.145	TCP	1304	443 → 49922 [PSH, ACK] Seq=1 Ack=213 Win=64240 Len=1250 [TCP
573	17.217689400	20.42.73.26	192.168.100.145	TCP	1304	443 → 49922 [PSH, ACK] Seq=1251 Ack=213 Win=64240 Len=1250 [
574	17.217782500	192.168.100.145	20.42.73.26	TCP	54	49922 → 443 [ACK] Seq=213 Ack=2501 Win=64240 Len=0
575	17.880548800	20.42.73.26	192.168.100.145	TCP	1514	443 → 49922 [ACK] Seq=2501 Ack=213 Win=64240 Len=1460 [TCP P
576	17.880553500	20.42.73.26	192.168.100.145	TLSv1.2	386	Server Hello, Certificate, Server Key Exchange, Server Hello
577	17.880737400	192.168.100.145	20.42.73.26	TCP	54	49922 → 443 [ACK] Seq=213 Ack=4293 Win=64240 Len=0
578	17.885033100	192.168.100.145	20.42.73.26	TLSv1.2	212	Client Key Exchange, Change Cipher Spec, Encrypted Handshake
579	17.885893900	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4293 Ack=371 Win=64240 Len=0
580	18.227276600	20.42.73.26	192.168.100.145	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
581	18.238860900	192.168.100.145	20.42.73.26	TLSv1.2	141	Application Data
582	18.239235300	192.168.100.145	20.42.73.26	TLSv1.2	950	Application Data
583	18.239609100	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4344 Ack=458 Win=64240 Len=0
584	18.239610000	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4344 Ack=1354 Win=64240 Len=0
585	18.239678500	192.168.100.145	20.42.73.26	TLSv1.2	2019	Application Data
586	18.240107200	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4344 Ack=2814 Win=64240 Len=0
587	18.240108500	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4344 Ack=3319 Win=64240 Len=0
588	18.272956800	20.42.73.26	192.168.100.145	TLSv1.2	123	Application Data
589	18.273325800	192.168.100.145	20.42.73.26	TLSv1.2	92	Application Data
590	18.273803500	20.42.73.26	192.168.100.145	TCP	60	443 → 49922 [ACK] Seq=4413 Ack=3357 Win=64240 Len=0
591	18.799029000	20.42.73.26	192.168.100.145	TLSv1.2	92	Application Data

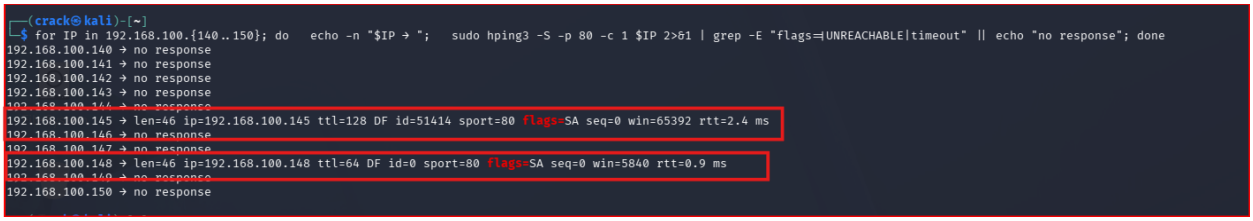
266	1.848394800	192.168.100.145	4.213.25.241	TLSv1.2	155	Application Data
267	1.849625200	4.213.25.241	192.168.100.145	TCP	60	443 → 49719 [ACK] Seq=1 Ack=102 Win=64240 Len=0
268	1.889961700	192.168.100.145	192.168.183.129	TCP	66	49920 → 9997 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SAC
269	1.905807600	MS-NLB-VirtServer_c…	Broadcast	ARP	42	Who has 192.168.100.75? Tell 192.168.100.145
270	1.977150300	4.213.25.241	192.168.100.145	TLSv1.2	225	Application Data
271	2.030878100	192.168.100.145	4.213.25.241	TCP	54	49719 → 443 [ACK] Seq=102 Ack=172 Win=64069 Len=0

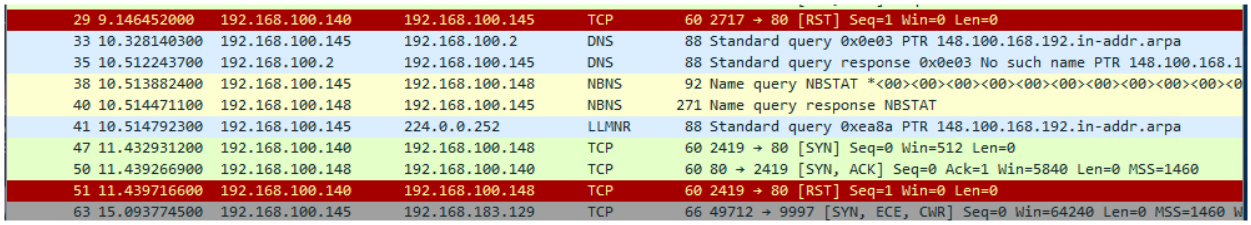
Finding: Packet trace confirmed ICMP Echo requests/replies and DNS PTR lookups occurring during the sweep, giving full visibility into what Nmap sends and receives at the packet level — useful for validating scan behavior against Wireshark captures.

Custom Discovery Script Development

Purpose: Combine multiple manual techniques into a single reusable script.

Multi-Technique Discovery Script (TCP probes via hping3)

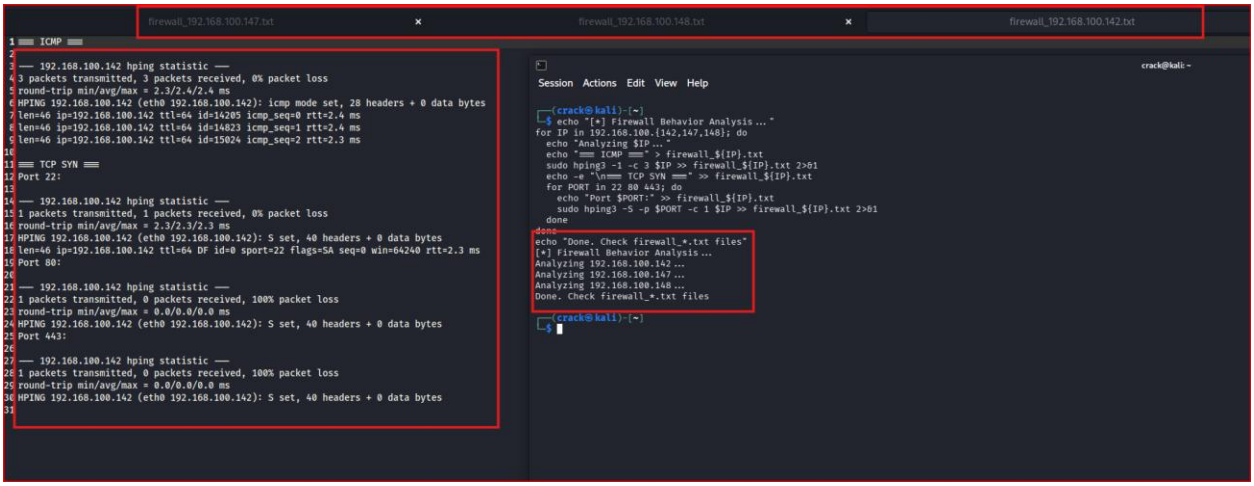




Finding: Only .145 (Windows) and .148 (Metasploitable) responded on port 80; all other IPs in the range returned no response, confirming only two hosts serve HTTP on this subnet.

Firewall Analysis — DVWA (192.168.100.142)

```
sudo hping3 -1 -c 3 192.168.100.142       # ICMP
sudo hping3 -S -p 22 -c 1 192.168.100.142 # SSH
sudo hping3 -S -p 80 -c 1 192.168.100.142 # HTTP
sudo hping3 -S -p 443 -c 1 192.168.100.142 # HTTPS
```



Finding / Interpretation:

- Host is **alive** — responds to ICMP
- Port 22 (SSH) is **open**
- Ports 80 and 443 are **blocked/filtered** — packets are silently dropped by a firewall (no RST, no ACK received)

Summary

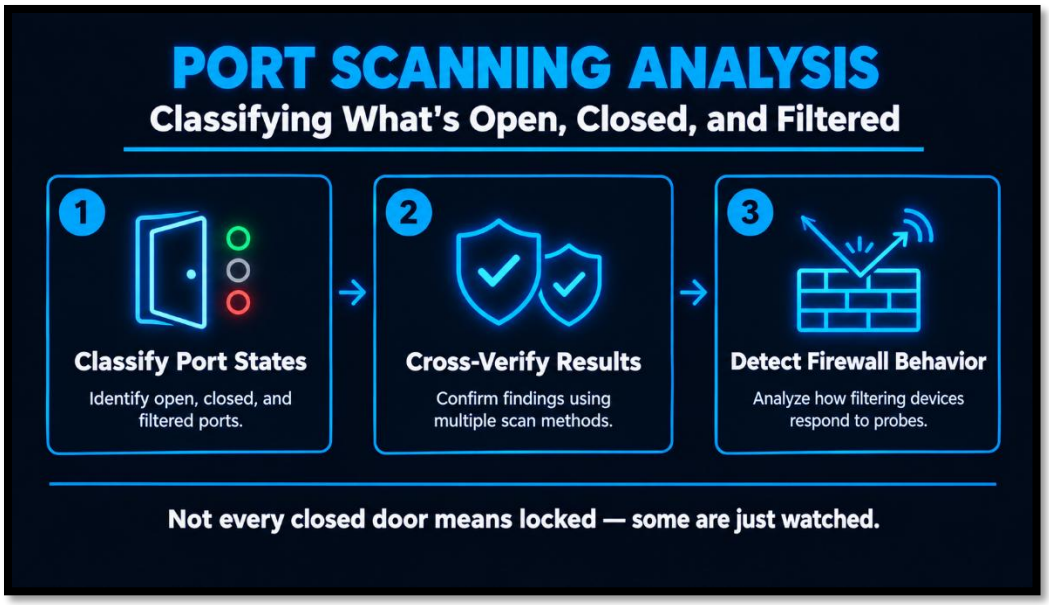
Technique	Purpose	Key Outcome
Custom TCP flag probing	Understand raw port state responses	Identified open vs. filtered ports without standard ping
Zombie/idle scanning	Scan without revealing real IP	Successfully scanned two targets via Windows zombie
ICMP/TCP filtering tests	Detect firewall rules	Distinguished open, closed, and filtered states per host
Timing/rate control	Balance speed vs. stealth	Rate-limited scan ≈12s vs. optimized scan ≈2.7s
TTL/topology analysis	Fingerprint OS and estimate hops	Linux hosts TTL 64, Windows host TTL 128
Custom scripting	Automate repeated discovery tasks	Built reusable multi-host, multi-port discovery script

---

Port Scanning Analysis

Objectives

- Master port state classification (all 6 states)
- Implement multi-method verification strategies
- Detect and analyze firewall filtering patterns
- Develop adaptive scanning based on network conditions
- Analyze target-specific port behavior
- Optimize scanning strategies per target



Focus: Advanced port scanning methodologies (scanning only — no service/version enumeration)

Target	IP Address
DVWA	192.168.100.142
Windows	192.168.100.145
Metasploitable	192.168.100.148

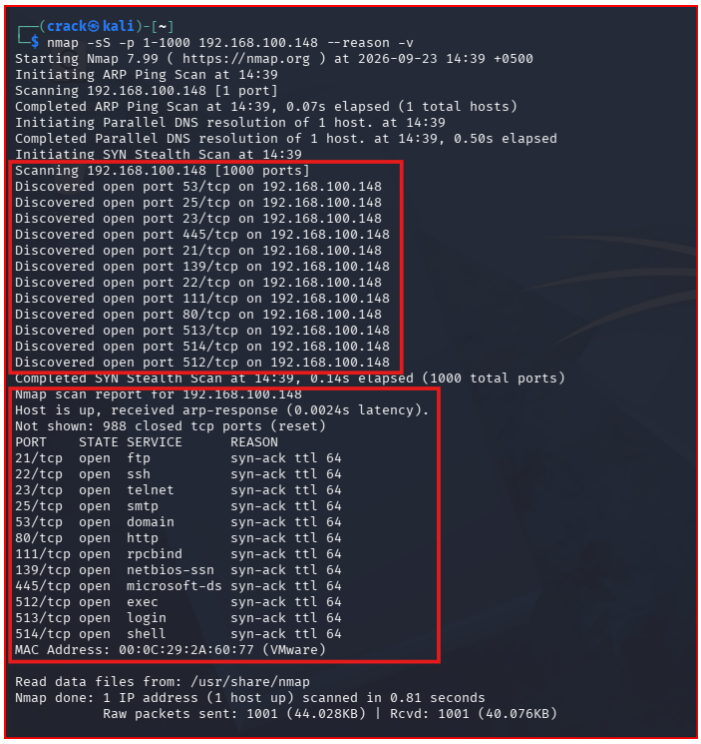
Deliverable: Comprehensive port analysis report with findings across all three machines.

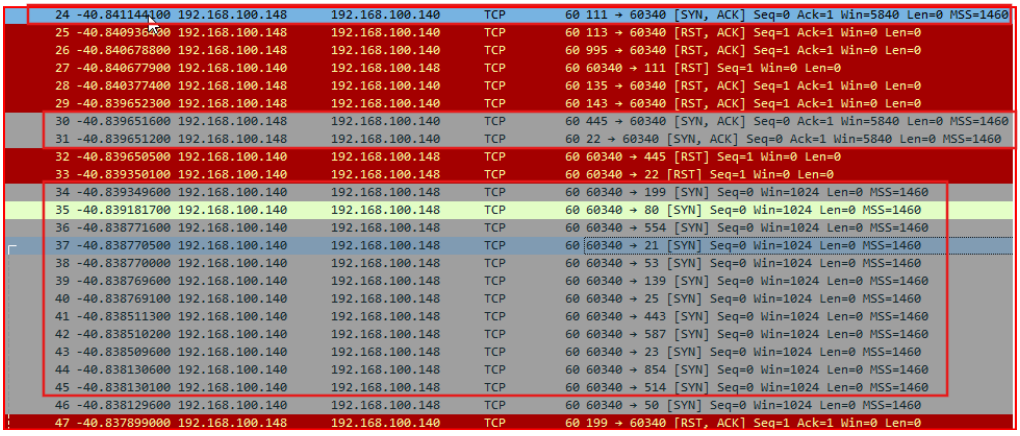
Port State Classification Mastery

Objective: Understand and correctly identify all six Nmap port states using multiple scan methods on the same target.

SYN Stealth Scan (Baseline)

```
nmap -sS -p 1-1000 192.168.100.148 --reason -v
```

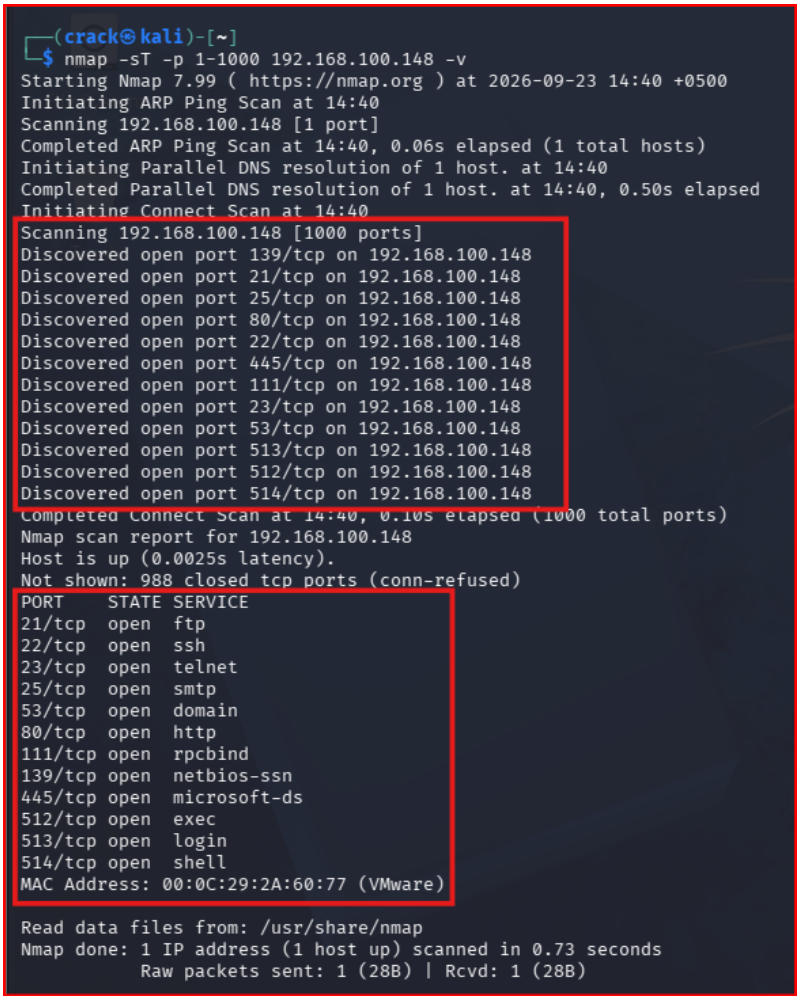




Finding: 11 open ports identified (21, 22, 23, 25, 53, 80, 111, 139, 445, 512, 513, 514), each confirmed via a SYN-ACK response with TTL 64 — a fast, low-footprint baseline scan.

TCP Connect Scan (Verification)

```
nmap -sT -p 1-1000 192.168.100.148 -v
```



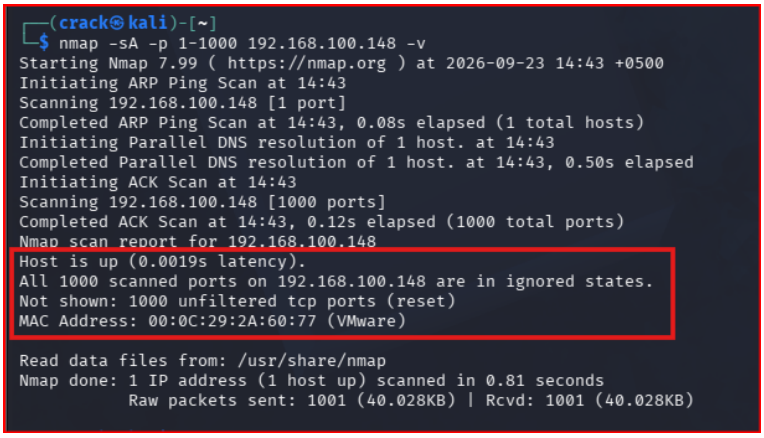
Both detected the exact same open ports.

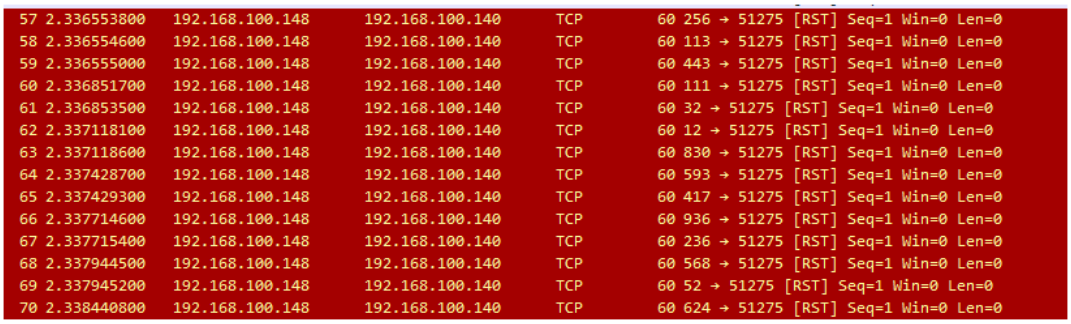
-sS is faster/stealthier and shows the reason; -sT is simpler but more detectable.

Finding: Identical open ports were detected as the SYN scan, confirming result accuracy. -sT completes the full TCP handshake and is more easily logged by the target, while -sS is faster and stealthier since it never completes the handshake.

ACK Scan (Firewall Testing)

```
nmap -sA -p 1-1000 192.168.100.148 -v
```





Finding: All 1000 ports returned RST regardless of actual open/closed state, so Nmap reported them as "unfiltered." This confirms **no stateful firewall** is present on this host — an ACK scan cannot determine open vs. closed on an unfiltered target, only whether a firewall is filtering traffic.

Scan Behavior Reference Table

Scan Type	Port Open	Port Closed	Port Filtered (Firewall)
-sS (SYN)	SYN-ACK	RST	No response
-sA (ACK)	RST	RST	No response

Target-Specific Scanning Analysis

Objective: Compare port behavior and exposure across all three lab targets.

Metasploitable (192.168.100.148) — Linux

```
nmap -sS --top-ports 100 192.168.100.148 -v
```

```
Scanning 192.168.100.148 [1 port]
Completed ARP Ping Scan at 14:44, 0.06s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 14:44
Completed Parallel DNS resolution of 1 host. at 14:44, 0.50s elapsed
Initiating SYN Stealth Scan at 14:44
Scanning 192.168.100.148 [100 ports]
Discovered open port 22/tcp on 192.168.100.148
Discovered open port 5900/tcp on 192.168.100.148
Discovered open port 3306/tcp on 192.168.100.148
Discovered open port 53/tcp on 192.168.100.148
Discovered open port 21/tcp on 192.168.100.148
Discovered open port 23/tcp on 192.168.100.148
Discovered open port 445/tcp on 192.168.100.148
Discovered open port 25/tcp on 192.168.100.148
Discovered open port 139/tcp on 192.168.100.148
Discovered open port 80/tcp on 192.168.100.148
Discovered open port 111/tcp on 192.168.100.148
Discovered open port 2049/tcp on 192.168.100.148
Discovered open port 513/tcp on 192.168.100.148
Discovered open port 8009/tcp on 192.168.100.148
Discovered open port 6000/tcp on 192.168.100.148
Discovered open port 5432/tcp on 192.168.100.148
Discovered open port 2121/tcp on 192.168.100.148
Discovered open port 514/tcp on 192.168.100.148
Completed SYN Stealth Scan at 14:44, 0.04s elapsed (100 total ports)
Nmap scan report for 192.168.100.148
Host is up (0.00025s latency).
Not shown: 82 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
513/tcp  open  login
514/tcp  open  shell
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
8009/tcp open  ajp13
MAC Address: 00:0C:29:2A:60:77 (VMware)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.72 seconds
           Raw packets sent: 101 (4.428KB) | Rcvd: 101 (4.100KB)
```



Findings:

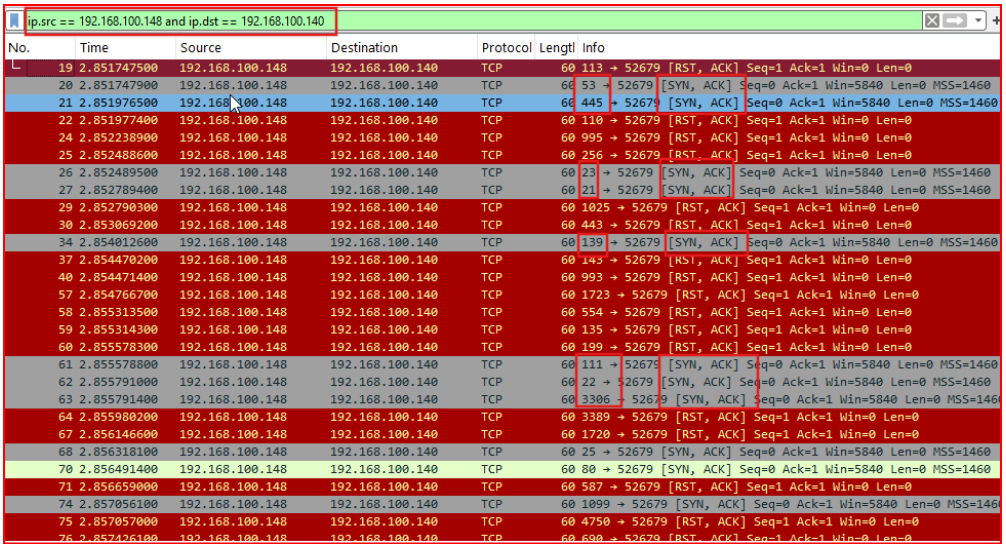
- Top 100 scan revealed 15 open TCP ports including FTP, SSH, Telnet, SMTP, DNS, HTTP, RPC, SMB, VNC, MySQL, PostgreSQL, and X11 — a heavily exposed host by design

---

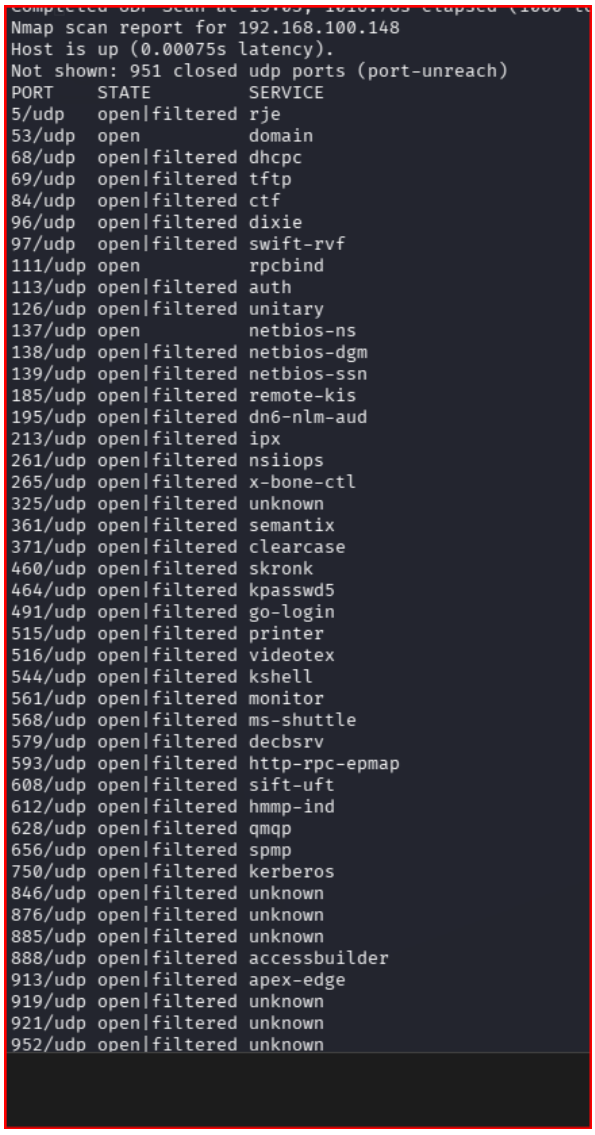
Extended scan (1–5000) added distccd (3632), rmiregistry (1099), and NFS (2049) to the open list

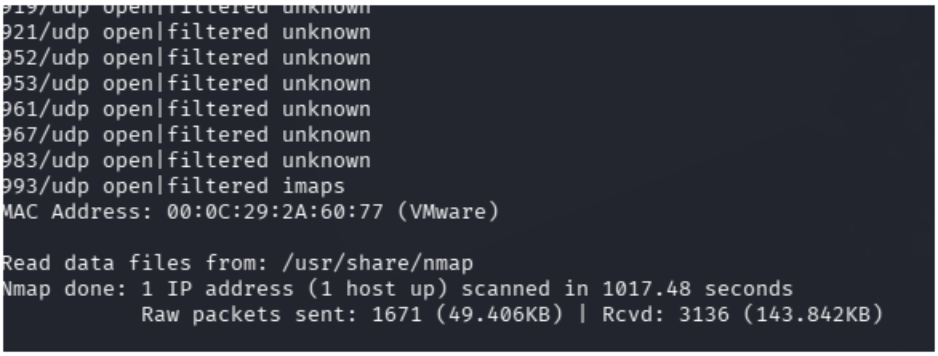
```
Scanning 192.168.100.148 [1 port]
Completed ARP Ping Scan at 14:45, 0.06s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 14:45
Completed Parallel DNS resolution of 1 host. at 14:45, 0.50s elapsed
Initiating SYN Stealth Scan at 14:45
Scanning 192.168.100.148 [5000 ports]
Discovered open port 139/tcp on 192.168.100.148
Discovered open port 3306/tcp on 192.168.100.148
Discovered open port 22/tcp on 192.168.100.148
Discovered open port 25/tcp on 192.168.100.148
Discovered open port 21/tcp on 192.168.100.148
Discovered open port 53/tcp on 192.168.100.148
Discovered open port 445/tcp on 192.168.100.148
Discovered open port 80/tcp on 192.168.100.148
Discovered open port 23/tcp on 192.168.100.148
Discovered open port 111/tcp on 192.168.100.148
Discovered open port 1524/tcp on 192.168.100.148
Discovered open port 2049/tcp on 192.168.100.148
Discovered open port 514/tcp on 192.168.100.148
Discovered open port 1099/tcp on 192.168.100.148
Discovered open port 2121/tcp on 192.168.100.148
Discovered open port 512/tcp on 192.168.100.148
Discovered open port 513/tcp on 192.168.100.148
Discovered open port 3632/tcp on 192.168.100.148
Completed SYN Stealth Scan at 14:45, 0.53s elapsed (5000 total ports)
Nmap scan report for 192.168.100.148
Host is up, received arp-response (0.0020s latency).
Not shown: 4982 closed tcp ports (reset)
PORT     STATE SERVICE      REASON
21/tcp   open  ftp          syn-ack ttl 64
22/tcp   open  ssh          syn-ack ttl 64
23/tcp   open  telnet       syn-ack ttl 64
25/tcp   open  smtp         syn-ack ttl 64
53/tcp   open  domain       syn-ack ttl 64
80/tcp   open  http         syn-ack ttl 64
111/tcp  open  rpcbind      syn-ack ttl 64
139/tcp  open  netbios-ssn  syn-ack ttl 64
445/tcp  open  microsoft-ds syn-ack ttl 64
512/tcp  open  exec         syn-ack ttl 64
513/tcp  open  login        syn-ack ttl 64
514/tcp  open  shell        syn-ack ttl 64
1099/tcp open  rmiregistry  syn-ack ttl 64
1524/tcp open  ingreslock   syn-ack ttl 64
2049/tcp open  nfs          syn-ack ttl 64
2121/tcp open  ccproxy-ftp  syn-ack ttl 64
3306/tcp open  mysql        syn-ack ttl 64
3632/tcp open  distccd      syn-ack ttl 64
MAC Address: 00:0C:29:2A:60:77 (VMware)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 1.20 seconds
           Raw packets sent: 5001 (220.028KB) | Rcvd: 5001 (200.100KB)
```



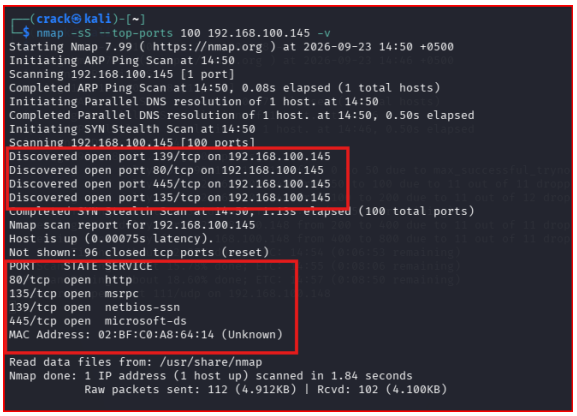
UDP scan (1–1000) returned a large number of open|filtered ports — expected behavior for UDP, since closed UDP ports return ICMP unreachable while open ones often produce no response at all, making UDP scanning inherently less precise than TCP

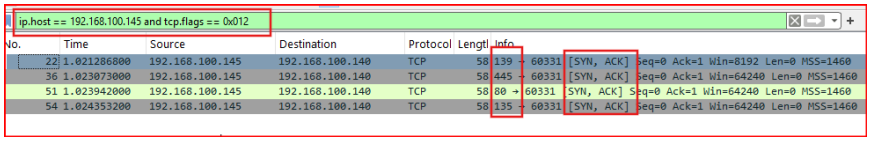




Windows (192.168.100.145)

```
nmap -sS --top-ports 100 192.168.100.145 -v
```

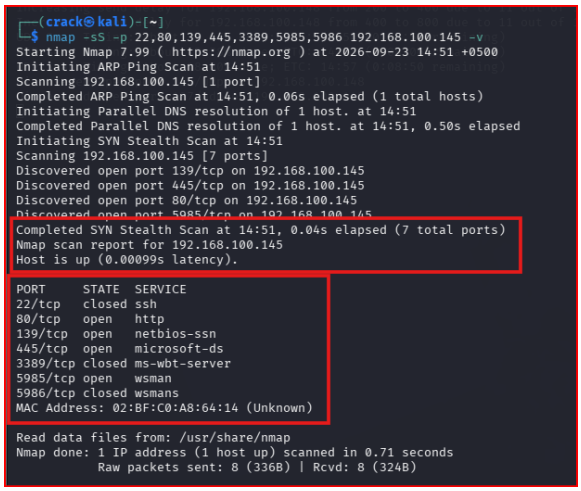


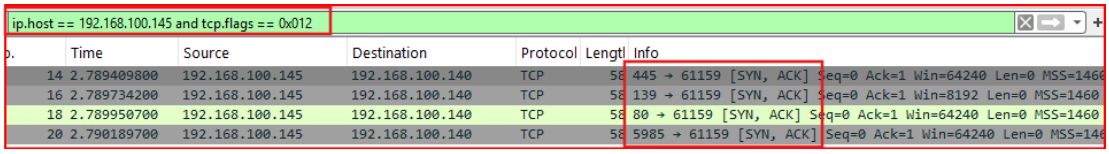


Findings: Top-100 scan showed only 4 open ports: 80 (HTTP), 135 (MSRPC), 139 (NetBIOS), 445 (SMB) — a much smaller attack surface than the Linux target

---

```
nmap -sS -p 22,80,139,445,3389,5985,5986 192.168.100.145 -v
```



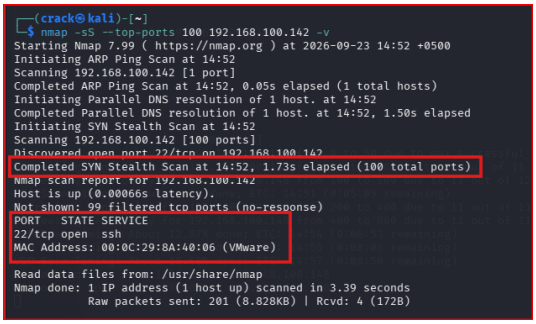


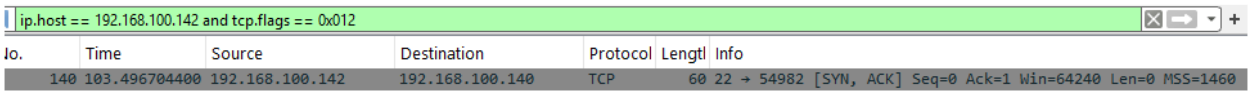
Findings: Targeted scan of common Windows management ports confirmed **3389 (RDP)** and **5986 (WinRM over HTTPS)** are **closed**, while **5985 (WinRM over HTTP)** is **open** — a specific and actionable finding for a Windows host profile

---

DVWA (192.168.100.142) Web Server

```
nmap -sS --top-ports 100 192.168.100.142 -v
```

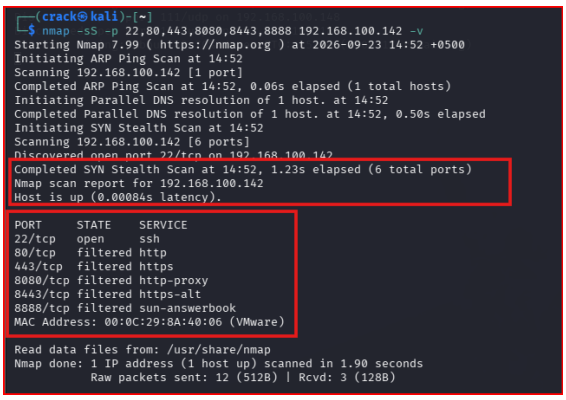


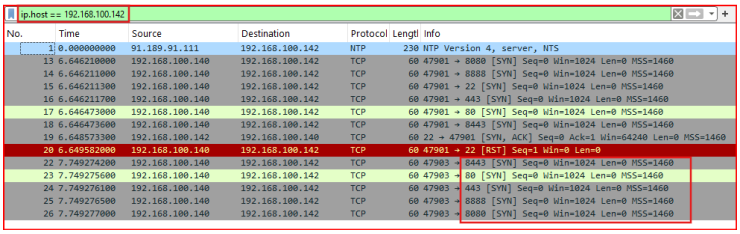


Findings: Top-100 scan showed only port 22 (SSH) open; all other top ports came back filtered (no response) rather than closed

---

```
nmap -sS -p 22,80,443,8080,8443,8888 192.168.100.142 -v
```





Findings: Targeted web-port scan confirmed port 22 open, while 80, 443, 8080, 8443, and 8888 were all **filtered** — indicating a firewall is actively dropping packets to web ports at the host/network level, despite DVWA actually running and reachable via the browser on 8080 in earlier setup steps. This is a strong sign of host-based firewall rules independent of the application layer.

Cross-Target Comparison

Target	Open Ports (Sample)	Notable Behavior
Metasploitable	15+ (FTP, SSH, Telnet, SMB, MySQL, etc.)	Broad, intentionally vulnerable exposure
Windows	4 (HTTP, MSRPC, NetBIOS, SMB)	Minimal exposure; RDP/WinRM-HTTPS closed
DVWA	1 (SSH only via scan)	Web ports filtered despite app being reachable directly

---

Firewall Pattern Detection

Objective: Identify firewall rules through systematic port response pattern analysis.

Automated Response Categorization

```
┌──(crack㉿kali)-[~]
└─$ #!/bin/bash
TARGET="192.168.100.148"

echo "Scanning ports to detect patterns..."
nmap -sS -p 1-1000 $TARGET --reason -oN port_analysis.txt

echo "Categorizing by response..."
echo "=== OPEN PORTS ==="
grep "open" port_analysis.txt | awk '{print $1}'

echo "=== CLOSED PORTS ==="
grep "closed" port_analysis.txt | awk '{print $1}' | head -5   # only show a few

echo "=== FILTERED PORTS ==="
grep "filtered" port_analysis.txt | awk '{print $1}'
```

```
Scanning ports to detect patterns...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 14:54 +0500
Nmap scan report for 192.168.100.148
Host is up, received arp-response (0.0023s latency).
Not shown: 988 closed tcp ports (reset)
PORT    STATE SERVICE     REASON
21/tcp  open  ftp         syn-ack ttl 64
22/tcp  open  ssh         syn-ack ttl 64
23/tcp  open  telnet      syn-ack ttl 64
25/tcp  open  smtp        syn-ack ttl 64
53/tcp  open  domain      syn-ack ttl 64
80/tcp  open  http        syn-ack ttl 64
111/tcp open  rpcbind     syn-ack ttl 64
139/tcp open  netbios-ssn syn-ack ttl 64
445/tcp open  microsoft-ds syn-ack ttl 64
512/tcp open  exec        syn-ack ttl 64
513/tcp open  login       syn-ack ttl 64
514/tcp open  shell       syn-ack ttl 64
MAC Address: 00:0C:29:2A:60:77 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.83 seconds
Categorizing by response...
=== OPEN PORTS ===
21/tcp
22/tcp
23/tcp
25/tcp
53/tcp
80/tcp
111/tcp
139/tcp
445/tcp
512/tcp
513/tcp
514/tcp
=== CLOSED PORTS ===
Not
=== FILTERED PORTS ===
```

ip.host == 192.168.100.148 and tcp.flags==0x012

No.	Time	Source	Destination	Protocol	Length	Info
16	1.938693200	192.168.100.148	192.168.100.140	TCP	60	111 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
23	1.939875000	192.168.100.148	192.168.100.140	TCP	60	445 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
26	1.940223100	192.168.100.148	192.168.100.140	TCP	60	25 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
27	1.940223900	192.168.100.148	192.168.100.140	TCP	60	139 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
47	1.942568500	192.168.100.148	192.168.100.140	TCP	60	80 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
52	1.942985600	192.168.100.148	192.168.100.140	TCP	60	22 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
54	1.942986300	192.168.100.148	192.168.100.140	TCP	60	23 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
55	1.943338000	192.168.100.148	192.168.100.140	TCP	60	53 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
57	1.943340100	192.168.100.148	192.168.100.140	TCP	60	21 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
374	2.018505500	192.168.100.148	192.168.100.140	TCP	60	514 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
1484	2.105098000	192.168.100.148	192.168.100.140	TCP	60	512 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
1739	2.123572500	192.168.100.148	192.168.100.140	TCP	60	513 → 34999 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460

Finding: Script successfully separated results into open, closed, and filtered categories automatically, confirming Metasploitable has 11 open ports and no filtered ports in the 1–1000 range — i.e., a completely unfiltered host from the network's perspective.

Adaptive Scanning Strategies

Objective: Dynamically adjust scan speed/method based on observed target behavior.

Target Responsiveness Assessment Script

A custom script (assess_target.sh) was built to:

1. Run a quick top 100 port scan
2. Count open vs. filtered ports
3. Recommend a scan strategy based on the ratio

```
  GNU nano 9.2
#!/bin/bash
# assess_target.sh

TARGET=$1

if [ -z "$TARGET" ]; then
  echo "Usage: $0 <target_ip>"
  exit 1
fi

echo "== QUICK ASSESSMENT OF $TARGET =="

echo "[Phase 1] Testing top 100 ports..."
START=$(date +%s)

nmap --top-ports 100 -sS $TARGET -oG phase1.txt 2>/dev/null

END=$(date +%s)
TIME1=$((END - START))

OPEN=$(grep -o "open" phase1.txt | wc -l)
FILTERED=$(grep -o "filtered" phase1.txt | wc -l)

echo "Results: $OPEN open, $FILTERED filtered in ${TIME1}s"

if [ $FILTERED -gt 50 ]; then
  echo "Decision: Heavy filtering detected - use slow scan"
  echo "Recommendation: -T1 or -T2 with increased timeouts"
elif [ $OPEN -gt 20 ]; then
  echo "Decision: Many open ports - full range scan recommended"
  echo "Recommendation: Continue with full port scan"
else
  echo "Decision: Balanced - standard scan is safe"
  echo "Recommendation: -T3 is appropriate"
fi
```

```
┌──(crack㉿kali)-[~]
└─$ nano assess_target.sh

┌──(crack㉿kali)-[~]
└─$ chmod +x assess_target.sh
sudo ./assess_target.sh 192.168.100.148
== QUICK ASSESSMENT OF 192.168.100.148 ==
[Phase 1] Testing top 100 ports...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:01 +0500
Nmap scan report for 192.168.100.148
Host is up (0.0012s latency).
Not shown: 82 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
513/tcp  open  login
514/tcp  open  shell
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
8009/tcp open  ajp13
MAC Address: 00:0C:29:2A:60:77 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.75 seconds
Results: 18 open, 0 filtered in 1s
Decision: Balanced - standard scan is safe
Recommendation: -T3 is appropriate

┌──(crack㉿kali)-[~]
└─$
```

ip.host == 192.168.100.148

No.	Time	Source	Destination	Protocol	Lengtl	Info
81	21.341594900	192.168.100.148	192.168.100.140	TCP	60	25 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
84	21.341873800	192.168.100.148	192.168.100.140	TCP	60	21 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
97	21.343321700	192.168.100.148	192.168.100.140	TCP	60	22 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
102	21.344355100	192.168.100.148	192.168.100.140	TCP	60	111 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
115	21.344899400	192.168.100.148	192.168.100.140	TCP	60	53 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
127	21.345893300	192.168.100.148	192.168.100.140	TCP	60	3306 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
128	21.346123200	192.168.100.148	192.168.100.140	TCP	60	139 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
133	21.346585200	192.168.100.148	192.168.100.140	TCP	60	445 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
136	21.347272000	192.168.100.148	192.168.100.140	TCP	60	23 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
138	21.347273100	192.168.100.148	192.168.100.140	TCP	60	5900 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
140	21.347756600	192.168.100.148	192.168.100.140	TCP	60	80 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
167	21.350265800	192.168.100.148	192.168.100.140	TCP	60	6000 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
168	21.350266200	192.168.100.148	192.168.100.140	TCP	60	2121 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
219	21.355129700	192.168.100.148	192.168.100.140	TCP	60	5432 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
227	21.356280000	192.168.100.148	192.168.100.140	TCP	60	514 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
249	21.359108000	192.168.100.148	192.168.100.140	TCP	60	2049 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
272	21.361201300	192.168.100.148	192.168.100.140	TCP	60	513 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
285	21.362815900	192.168.100.148	192.168.100.140	TCP	60	8009 → 59231 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460

Finding: Against Metasploitable, the script measured 18 open ports and 0 filtered, and correctly recommended a **standard -T3 scan** as appropriate — validating the decision logic against a real target.

Adaptive Scan Execution

A second script (adaptive_scan.sh) ran a two-phase scan: a quick top-100 assessment followed by a full 1–10000 port scan, with timing (-T1 vs -T3) automatically selected based on the Phase 1 filtering ratio.

```
#!/bin/bash
# adaptive_scan.sh

TARGET="192.168.100.148"

echo "[Phase 1] Quick assessment ... "
nmap --top-ports 100 -sS $TARGET -oG phase1.txt

FILTERED=$(grep -o "filtered" phase1.txt | wc -l)

if [ $FILTERED -gt 50 ]; then
  echo "[Phase 2] Heavy filtering detected → using slow scan (-T1)"
  nmap -sS -T1 -p 1-10000 $TARGET -oG phase2.txt
else
  echo "[Phase 2] Normal conditions → using standard scan (-T3)"
  nmap -sS -T3 -p 1-10000 $TARGET -oG phase2.txt
fi

echo "[Complete] Results saved in phase1.txt and phase2.txt"
```

```
└─$ chmod +x adaptive_scan.sh
└─$ ./adaptive_scan.sh
[Phase 1] Quick assessment ...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:03 +0500
Nmap scan report for 192.168.100.148
Host is up (0.00085s latency).
Not shown: 82 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
513/tcp  open  login
514/tcp  open  shell
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
8009/tcp open  ajp13
MAC Address: 00:0C:29:2A:60:77 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds
[Phase 2] Normal conditions → using standard scan (-T3)
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:03 +0500
Nmap scan report for 192.168.100.148
Host is up (0.0021s latency).
Not shown: 9974 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
```

ip.host == 192.168.100.148

No.	Time	Source	Destination	Protocol	Length	Info
17	2.41[illegible]4300	192.168.100.148	192.168.100.140	TCP	60	111 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
18	2.417845200	192.168.100.148	192.168.100.140	TCP	60	23 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
19	2.418103500	192.168.100.148	192.168.100.140	TCP	60	53 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
21	2.418486300	192.168.100.148	192.168.100.140	TCP	60	25 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
27	2.419288400	192.168.100.148	192.168.100.140	TCP	60	3306 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
40	2.421226700	192.168.100.148	192.168.100.140	TCP	60	21 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
50	2.422154000	192.168.100.148	192.168.100.140	TCP	60	5900 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
60	2.422962300	192.168.100.148	192.168.100.140	TCP	60	80 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
63	2.423285800	192.168.100.148	192.168.100.140	TCP	60	139 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
66	2.423558500	192.168.100.148	192.168.100.140	TCP	60	445 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
67	2.423558900	192.168.100.148	192.168.100.140	TCP	60	22 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
140	2.431496400	192.168.100.148	192.168.100.140	TCP	60	2049 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
153	2.433619900	192.168.100.148	192.168.100.140	TCP	60	8009 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
180	2.437183400	192.168.100.148	192.168.100.140	TCP	60	514 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
195	2.437831800	192.168.100.148	192.168.100.140	TCP	60	2121 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
197	2.437832300	192.168.100.148	192.168.100.140	TCP	60	5432 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
204	2.438173000	192.168.100.148	192.168.100.140	TCP	60	513 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
219	2.440674800	192.168.100.148	192.168.100.140	TCP	60	6000 → 39987 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
234	3.294552400	192.168.100.148	192.168.100.140	TCP	60	53 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
244	3.296095600	192.168.100.148	192.168.100.140	TCP	60	3306 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
247	3.296837900	192.168.100.148	192.168.100.140	TCP	60	21 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
249	3.297185400	192.168.100.148	192.168.100.140	TCP	60	111 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
265	3.300372800	192.168.100.148	192.168.100.140	TCP	60	23 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
277	3.301912300	192.168.100.148	192.168.100.140	TCP	60	5900 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
280	3.302125600	192.168.100.148	192.168.100.140	TCP	60	80 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
287	3.303057900	192.168.100.148	192.168.100.140	TCP	60	139 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
288	3.303339000	192.168.100.148	192.168.100.140	TCP	60	22 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
293	3.303944600	192.168.100.148	192.168.100.140	TCP	60	445 → 57700 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460

Finding: Phase 1 detected 0% filtering, triggering the standard -T3 scan in Phase 2, which then expanded the discovered port list from 18 to 22 open ports (adding distccd, IRC, and additional unknown services) — demonstrating that a deeper full-range scan can reveal ports missed by a top-ports-only sweep.

Statistical Analysis

Objective: Quantify port distribution across all targets for comparative reporting.

```
GNU nano 9.2
#!/bin/bash
# statistical_analysis.sh

echo "== Starting Statistical Analysis =="

for HOST in 192.168.100.{142,147,148}; do
  echo "Scanning $HOST..."
  nmap -sS -p 1-5000 $HOST -oG scan_$HOST.txt 2>/dev/null
done

echo ""
echo "== STATISTICAL COMPARISON =="

for FILE in scan_*.txt; do
  HOST=$(echo $FILE | grep -oP '\d+\.\d+\.\d+\.\d+')
  OPEN=$(grep -o "open" $FILE | wc -l)
  CLOSED=$(grep -o "closed" $FILE | wc -l)
  FILTERED=$(grep -o "filtered" $FILE | wc -l)
  TOTAL=$((OPEN + CLOSED + FILTERED))

  if [ $TOTAL -gt 0 ]; then
    OPEN_PCT=$((OPEN * 100 / TOTAL))
  else
    OPEN_PCT=0
  fi

  echo "$HOST → Open: $OPEN ($OPEN_PCT%) | Closed: $CLOSED | Filtered: $FILTERED"
done

echo ""
echo "[v] Analysis complete"
```

```
┌──(crack㉿kali)-[~]
└─$ nano statistical_analysis.sh

┌──(crack㉿kali)-[~]
└─$ chmod +x statistical_analysis.sh
sudo ./statistical_analysis.sh
[sudo] password for crack:
== Starting Statistical Analysis ==
Scanning 192.168.100.142 ...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:07 +0500
Nmap scan report for 192.168.100.142
Host is up (0.0011s latency).
Not shown: 4999 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 00:0C:29:8A:40:06 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 14.80 seconds
Scanning 192.168.100.147 ...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:07 +0500
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.49 seconds
Scanning 192.168.100.148 ...
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 15:07 +0500
Nmap scan report for 192.168.100.148
Host is up (0.0021s latency).
Not shown: 4982 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
3632/tcp open  distccd
MAC Address: 00:0C:29:2A:60:77 (VMware)

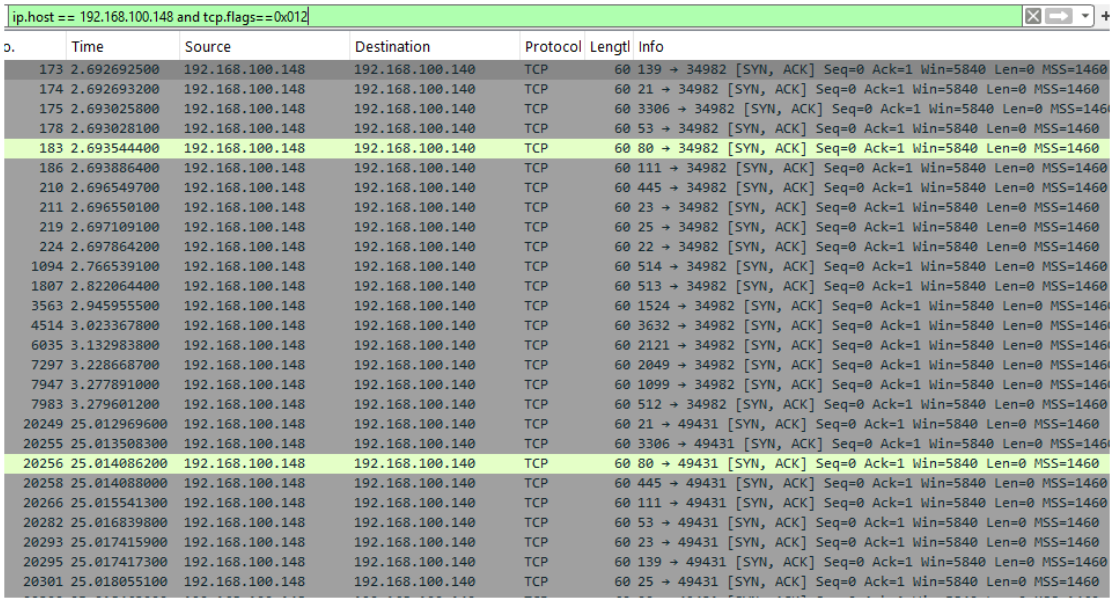
Nmap done: 1 IP address (1 host up) scanned in 1.29 seconds
== STATISTICAL COMPARISON ==
192.168.100.142 → Open: 1 (50%) | Closed: 0 | Filtered: 1
192.168.100.147 → Open: 0 (0%) | Closed: 0 | Filtered: 0
192.168.100.148 → Open: 18 (94%) | Closed: 1 | Filtered: 0
[v] Analysis complete
```

Finding — Statistical Comparison:

Host	Open	Open %	Closed	Filtered
192.168.100.142 (DVWA)	1	50%	0	1
192.168.100.147 (unused/down)	0	0%	0	0
192.168.100.148 (Metasploitable)	18	94%	1	0

```
77 0.083044500   192.168.100.140   192.168.100.142   TCP   60 46546 → 22 [RST] Seq=1 Win=0 Len=0
78 0.085787100   192.168.100.140   192.168.100.142   TCP   60 46523 → 3120 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
```

```
195 2.695150700   192.168.100.148   192.168.100.140   TCP   60 256 → 34982 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
```



ip.host == 192.168.100.148 and tcp.flags==0x012

o.	Time	Source	Destination	Protocol	Lengtl	Info
173	2.692692500	192.168.100.148	192.168.100.140	TCP	60	139 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
174	2.692693200	192.168.100.148	192.168.100.140	TCP	60	21 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
175	2.693025800	192.168.100.148	192.168.100.140	TCP	60	3306 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
178	2.693028100	192.168.100.148	192.168.100.140	TCP	60	53 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
183	2.693544400	192.168.100.148	192.168.100.140	TCP	60	80 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
186	2.693886400	192.168.100.148	192.168.100.140	TCP	60	111 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
210	2.696549700	192.168.100.148	192.168.100.140	TCP	60	445 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
211	2.696550100	192.168.100.148	192.168.100.140	TCP	60	23 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
219	2.697109100	192.168.100.148	192.168.100.140	TCP	60	25 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
224	2.697864200	192.168.100.148	192.168.100.140	TCP	60	22 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
1094	2.766539100	192.168.100.148	192.168.100.140	TCP	60	514 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
1807	2.822064400	192.168.100.148	192.168.100.140	TCP	60	513 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
3563	2.945955500	192.168.100.148	192.168.100.140	TCP	60	1524 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
4514	3.023367800	192.168.100.148	192.168.100.140	TCP	60	3632 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
6035	3.132983800	192.168.100.148	192.168.100.140	TCP	60	2121 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
7297	3.228668700	192.168.100.148	192.168.100.140	TCP	60	2049 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
7947	3.277891000	192.168.100.148	192.168.100.140	TCP	60	1099 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
7983	3.279601200	192.168.100.148	192.168.100.140	TCP	60	512 → 34982 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20249	25.012969600	192.168.100.148	192.168.100.140	TCP	60	21 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20255	25.013508300	192.168.100.148	192.168.100.140	TCP	60	3306 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=146
20256	25.014086200	192.168.100.148	192.168.100.140	TCP	60	80 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20258	25.014088000	192.168.100.148	192.168.100.140	TCP	60	445 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20266	25.015541300	192.168.100.148	192.168.100.140	TCP	60	111 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20282	25.016839800	192.168.100.148	192.168.100.140	TCP	60	53 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20293	25.017415900	192.168.100.148	192.168.100.140	TCP	60	23 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20295	25.017417300	192.168.100.148	192.168.100.140	TCP	60	139 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
20301	25.018055100	192.168.100.148	192.168.100.140	TCP	60	25 → 49431 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460

Note: 192.168.100.147 returned no response ("host seems down, blocking ping probes") — this IP was not part of the active lab range (the Windows host is actually .145), which explains the null result and should be corrected in future scan target lists.

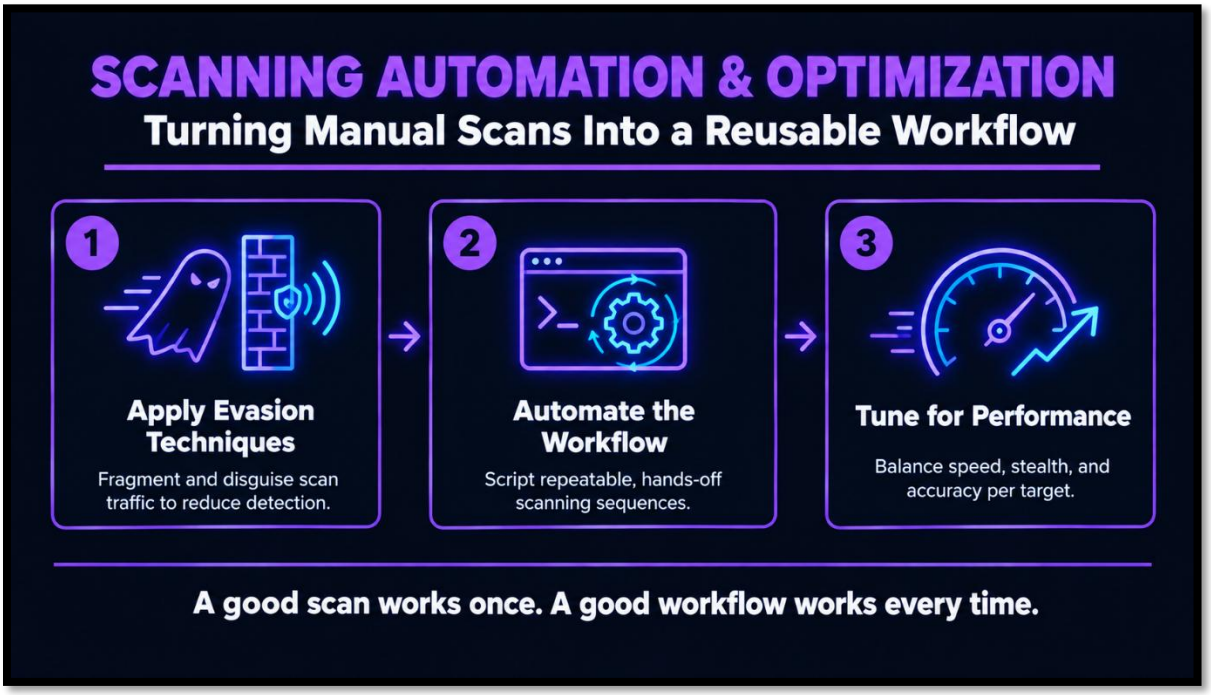
Summary

Technique	Purpose	Key Outcome
SYN vs. Connect vs. ACK scans	Validate port states across methods	Confirmed consistent open-port results; no stateful firewall on Metasploitable
Target-specific scanning	Profile each host individually	Windows: minimal exposure; Metasploitable: broad exposure; DVWA: web ports filtered
Firewall pattern detection	Automate open/closed/filtered classification	Metasploitable confirmed fully unfiltered in 1–1000 range
Adaptive scanning scripts	Auto-select scan speed based on filtering level	Correctly chose -T3 for an unfiltered target; deeper scan revealed 4 additional ports
Statistical comparison	Cross-host reporting	Metasploitable 94% open vs. DVWA 50% open (1 port) vs. undefined host 0%

Scanning Automation & Optimization

Objectives

- Integrate all scanning techniques from Projects 01 & 02
- Implement advanced evasion during scanning
- Develop custom automation scripts
- Optimize scanning performance
- Create professional scanning workflows
- Build reusable scanning tools
- Master timing and rate control strategies



Focus: Automation, optimization, and integration of advanced scanning techniques (scanning only — no enumeration)

Outcome: A professional, reusable scanning toolkit and workflow

Deliverable: Automated scanning suite with accompanying documentation

---

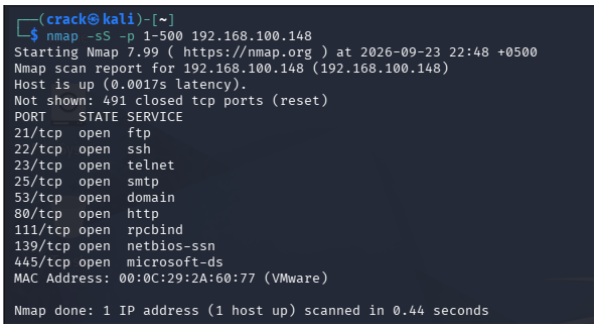
Advanced Evasion During Scanning

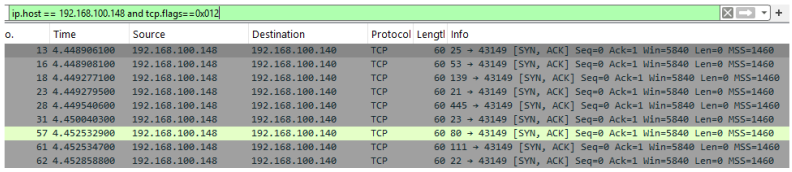
Objective: Apply evasion techniques from Project 01 while performing port scanning and measure their stealth/performance trade-offs.

Fragmented Port Scanning

Baseline — Standard SYN Scan

```
nmap -sS -p 1-500 192.168.100.148
```

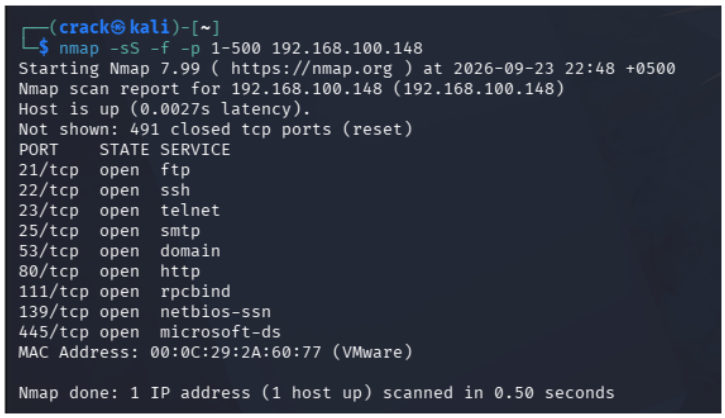


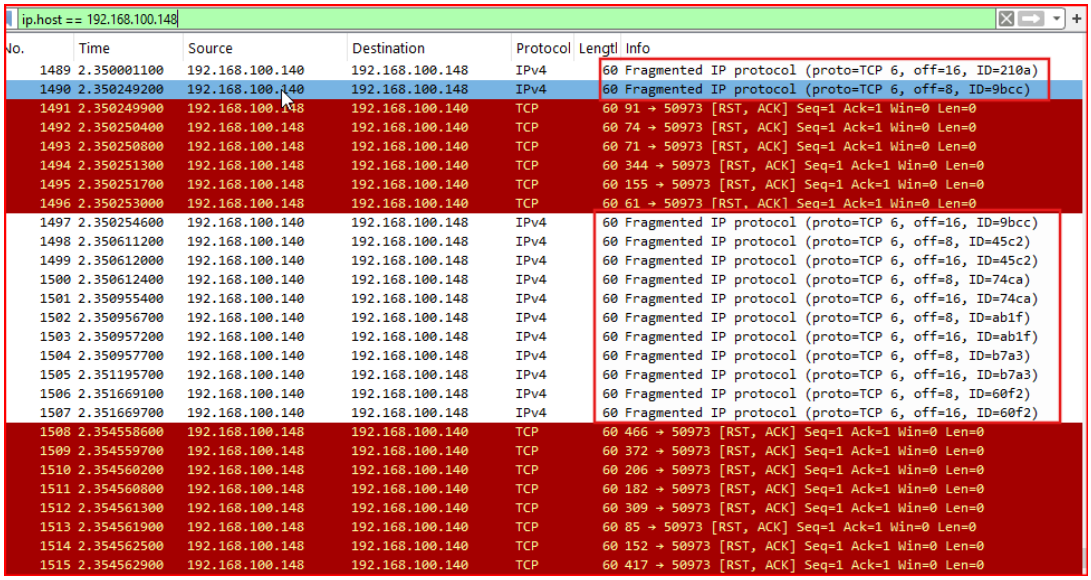


Finding: Completed in 0.44 seconds with 9 open ports identified. This is the fastest method but sends full, unmodified packets — easiest for a firewall/IDS to detect and log.

Fragmented Packets (Medium Stealth)

```
nmap -sS -f -p 1-500 192.168.100.148
```





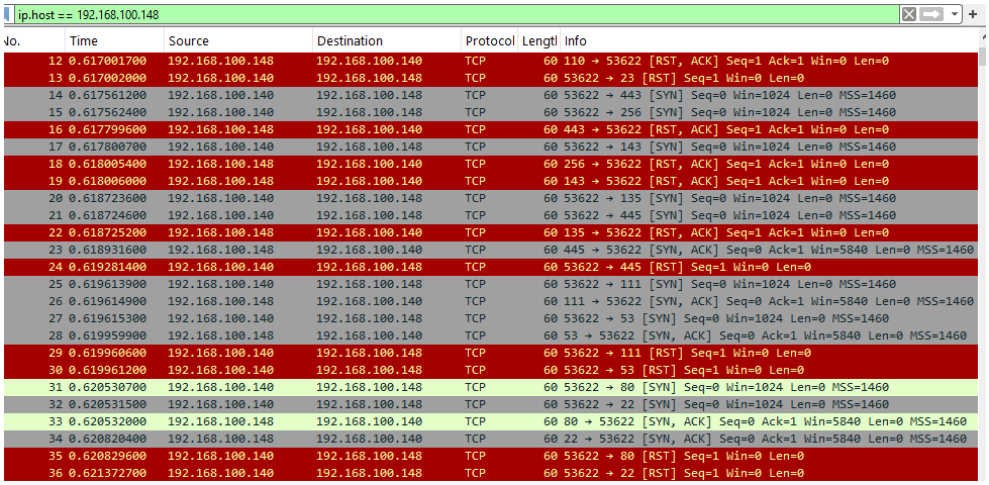
Finding: Same 9 ports identified, but scan time increased to 0.50 seconds. Splitting packets into fragments makes signature-based detection harder, at a small performance cost.

Extreme Fragmentation (High Stealth)

`nmap -sS --mtu 24 -p 1-500 192.168.100.148`

```
┌──(crack㉿kali)-[~]
└─$ nmap -sS --mtu 24 -p 1-500 192.168.100.148
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-23 22:49 +0500
Nmap scan report for 192.168.100.148 (192.168.100.148)
Host is up (0.0022s latency).
Not shown: 491 closed tcp ports (reset)
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
23/tcp  open  telnet
25/tcp  open  smtp
53/tcp  open  domain
80/tcp  open  http
111/tcp open  rpcbind
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:2A:60:77 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.44 seconds
```



Finding: Identical results (9 open ports) in 0.44 seconds. Using a very small MTU produces heavily fragmented packets, which further reduces detection risk with no measurable accuracy loss on this target.

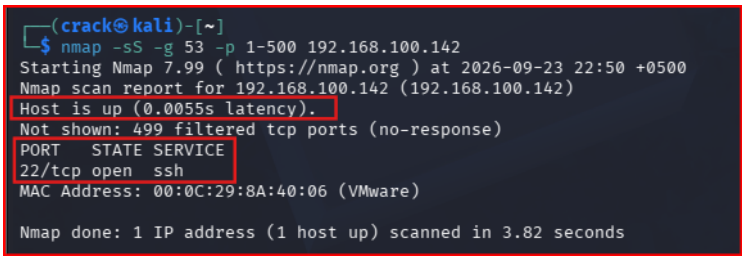
Comparison Table

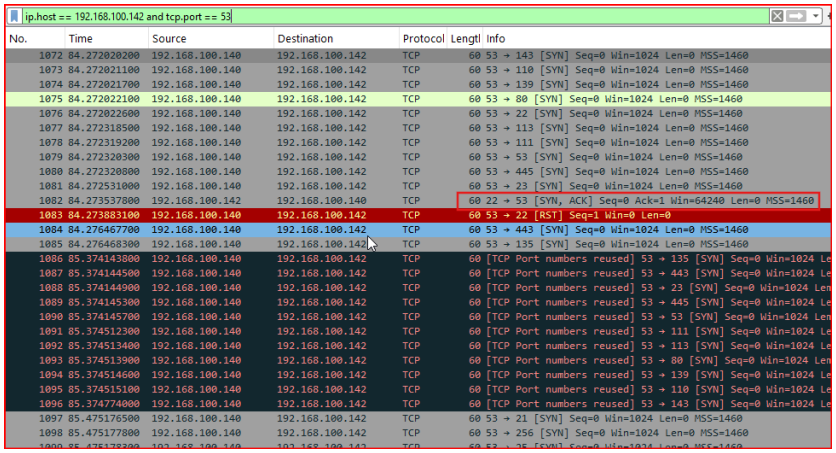
Technique	Stealth Level	Detection Risk	Result Accuracy
Standard SYN	Low	High	Baseline (9 open ports)
Fragmented (-f)	Medium	Medium	Identical to baseline
Extreme fragmentation (--mtu 24)	High	Low	Identical to baseline

Source Port Manipulation

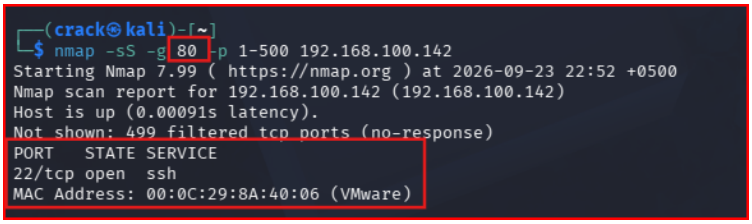
Purpose: Spoof the scan's source port to a commonly trusted port number (DNS/HTTP/HTTPS), exploiting firewalls that trust traffic from specific ports.

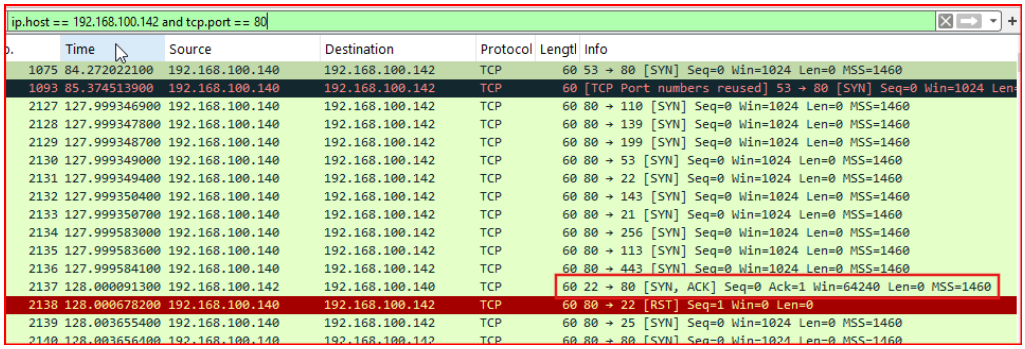
```
nmap -sS -g 53 -p 1-500 192.168.100.142     # Source port 53 (DNS)
```



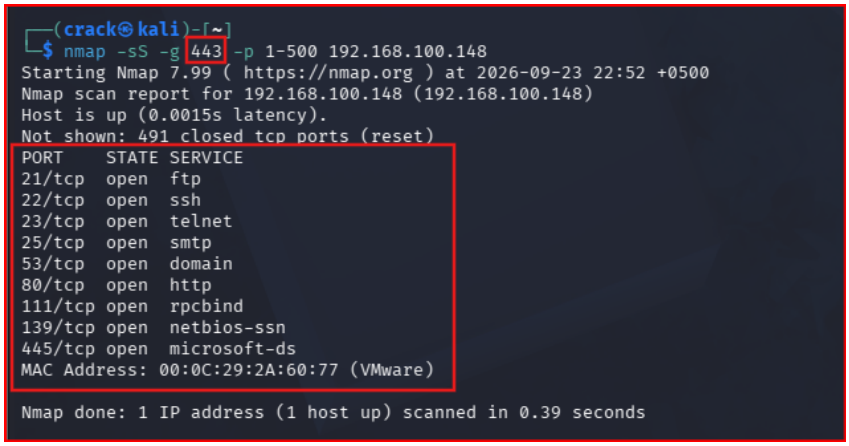


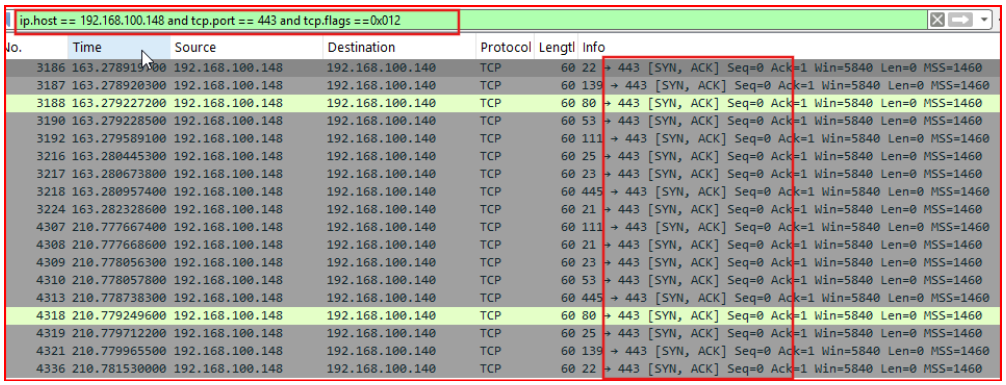

```
nmap -sS -g 80 -p 1-500 192.168.100.142     # Source port 80 (HTTP)
```





```
nmap -sS -g 443 -p 1-500 192.168.100.148    # Source port 443 (HTTPS)
```





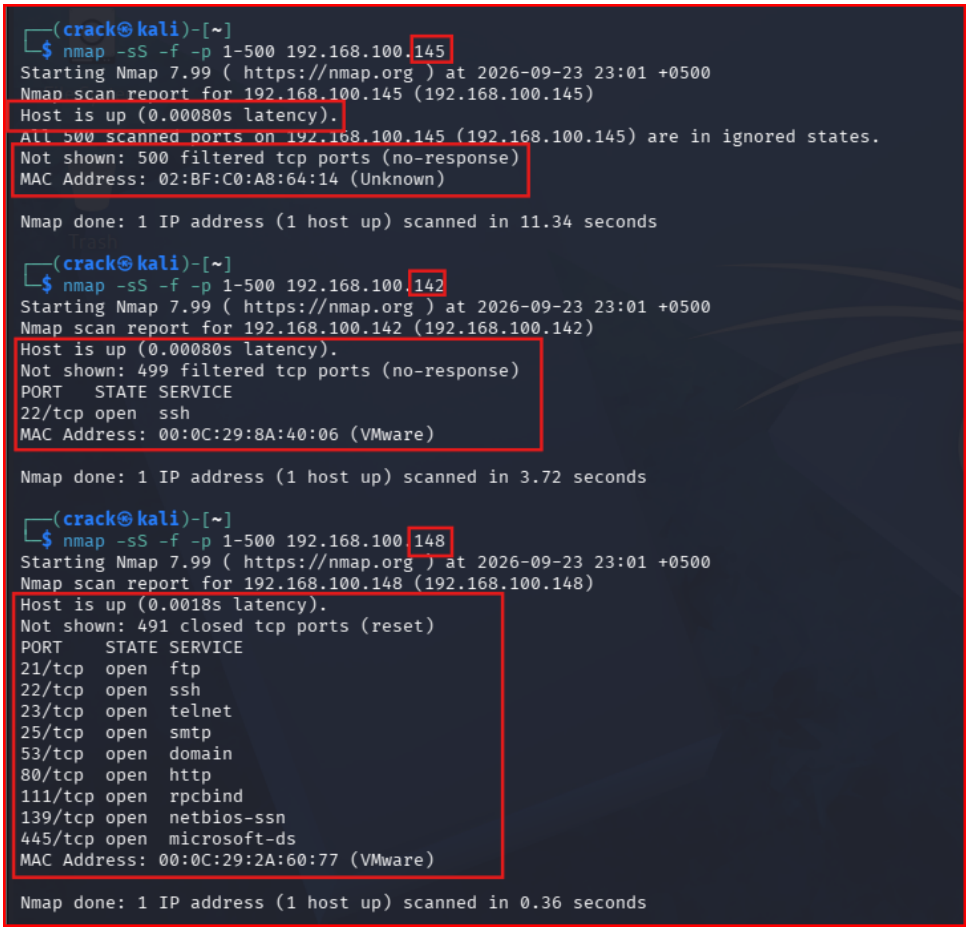
Finding: On DVWA (192.168.100.142), spoofing source port 53 or 80 did **not** bypass the existing filtering — only port 22 was returned as open in both cases, with 499 ports filtered. This indicates the firewall protecting DVWA is filtering based on destination port/state rather than trusting specific source ports.

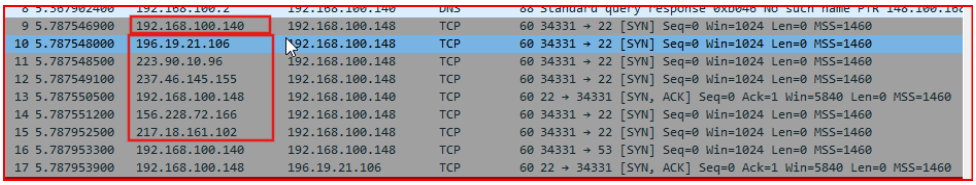
Decoy Scanning

Purpose: Mask the real scanning source among multiple decoy IP addresses, making it harder for a defender to identify the true attacker.

Random Decoys

```
nmap -sS -D RND:5 -p 1-500 192.168.100.148
```

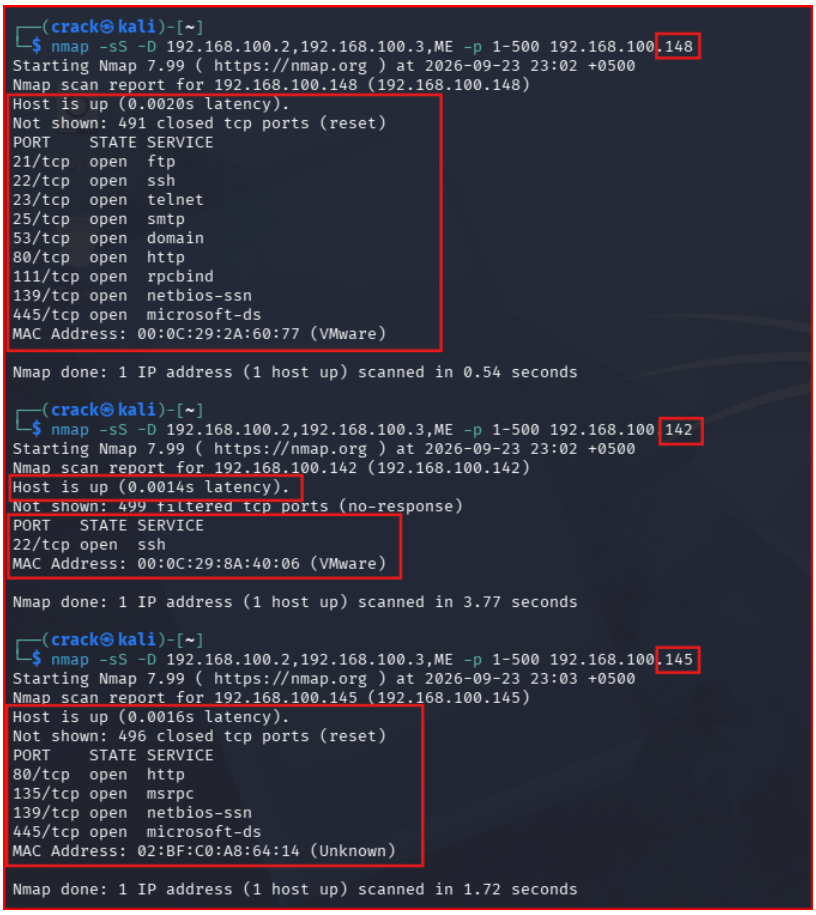


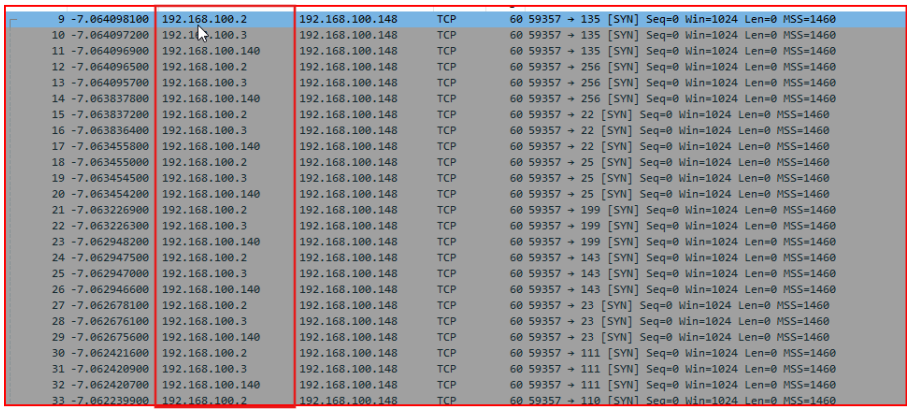


Finding: Scan completed successfully with identical results to the direct scan, while packet captures showed traffic appearing to originate from 5 additional randomized IP addresses alongside the real attacker — successfully obscuring the true source in traffic logs.

Specific Decoys

```
nmap -sS -D 192.168.100.2,192.168.100.3,ME -p 1-500
192.168.100.148
```



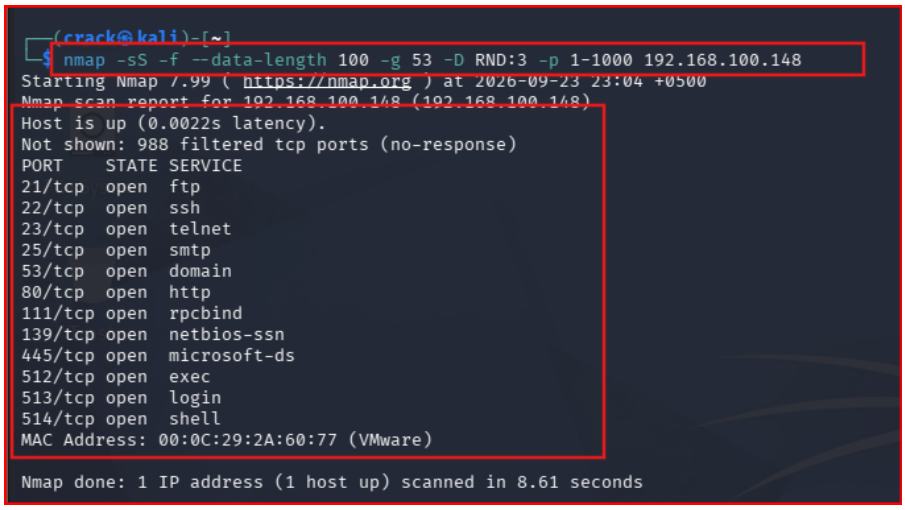


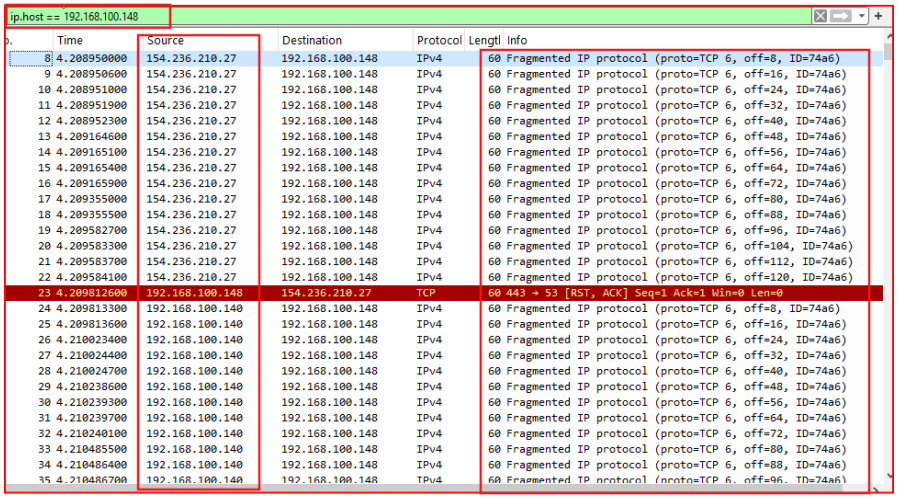
Finding: Using named decoy IPs (with ME marking the real attacker's position in the sequence) produced the same accurate port results across all three targets, confirming decoy scanning does not compromise scan accuracy — only attribution.

Combined Evasion

Purpose: Stack fragmentation, source port spoofing, and decoys into a single scan for maximum stealth.

```
nmap -sS -f --data-length 100 -g 53 -D RND:3 -p 1-1000
192.168.100.148
```





Finding: Completed in 8.61 seconds — noticeably slower than any individual technique — but successfully identified all 11 previously confirmed open ports. This confirms that combining evasion techniques is functionally reliable, though it comes at a clear performance cost that must be weighed against the stealth requirement of a given engagement.

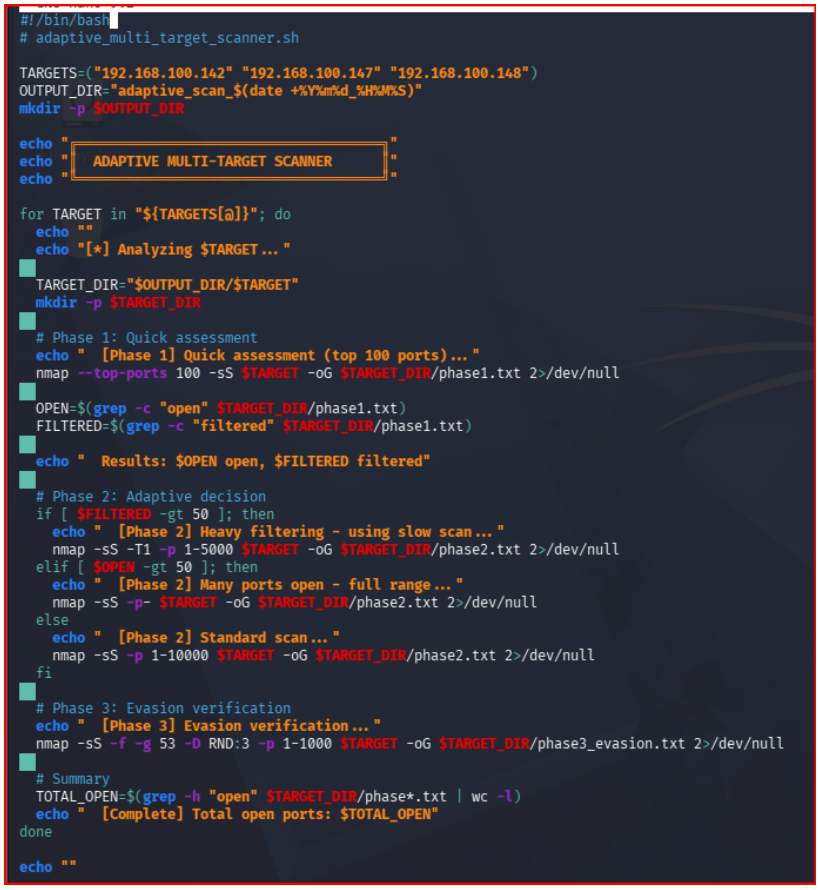
Custom Scanning Automation

Objective: Build reusable, decision-making scripts rather than running one-off manual commands.

Adaptive Multi-Target Scanner

A script (adaptive_multi_target_scanner.sh) was developed to automatically process a list of targets and, for each host:

1. Run a quick top-100 port assessment
2. Decide the next scan strategy based on the number of filtered ports:
    - Heavy filtering → slow scan (-T1, 1–5000)
    - Many open ports → full range scan (-p-)
    - Otherwise → standard scan (1–10000)
3. Run an evasion-verification pass (fragmented + spoofed source port + decoys)
4. Summarize total open ports found across all phases



```
#!/bin/bash
# adaptive_multi_target_scanner.sh

TARGETS=("192.168.100.142" "192.168.100.147" "192.168.100.148")
OUTPUT_DIR="adaptive_scan_$(date +%Y%m%d_%H%M%S)"
mkdir -p $OUTPUT_DIR

echo "                                    "
echo "   ADAPTIVE MULTI-TARGET SCANNER    "
echo "                                    "

for TARGET in "${TARGETS[@]}"; do
  echo ""
  echo "[*] Analyzing $TARGET..."

  TARGET_DIR="$OUTPUT_DIR/$TARGET"
  mkdir -p $TARGET_DIR

  # Phase 1: Quick assessment
  echo "  [Phase 1] Quick assessment (top 100 ports)..."
  nmap --top-ports 100 -sS $TARGET -oG $TARGET_DIR/phase1.txt 2>/dev/null

  OPEN=$(grep -c "open" $TARGET_DIR/phase1.txt)
  FILTERED=$(grep -c "filtered" $TARGET_DIR/phase1.txt)

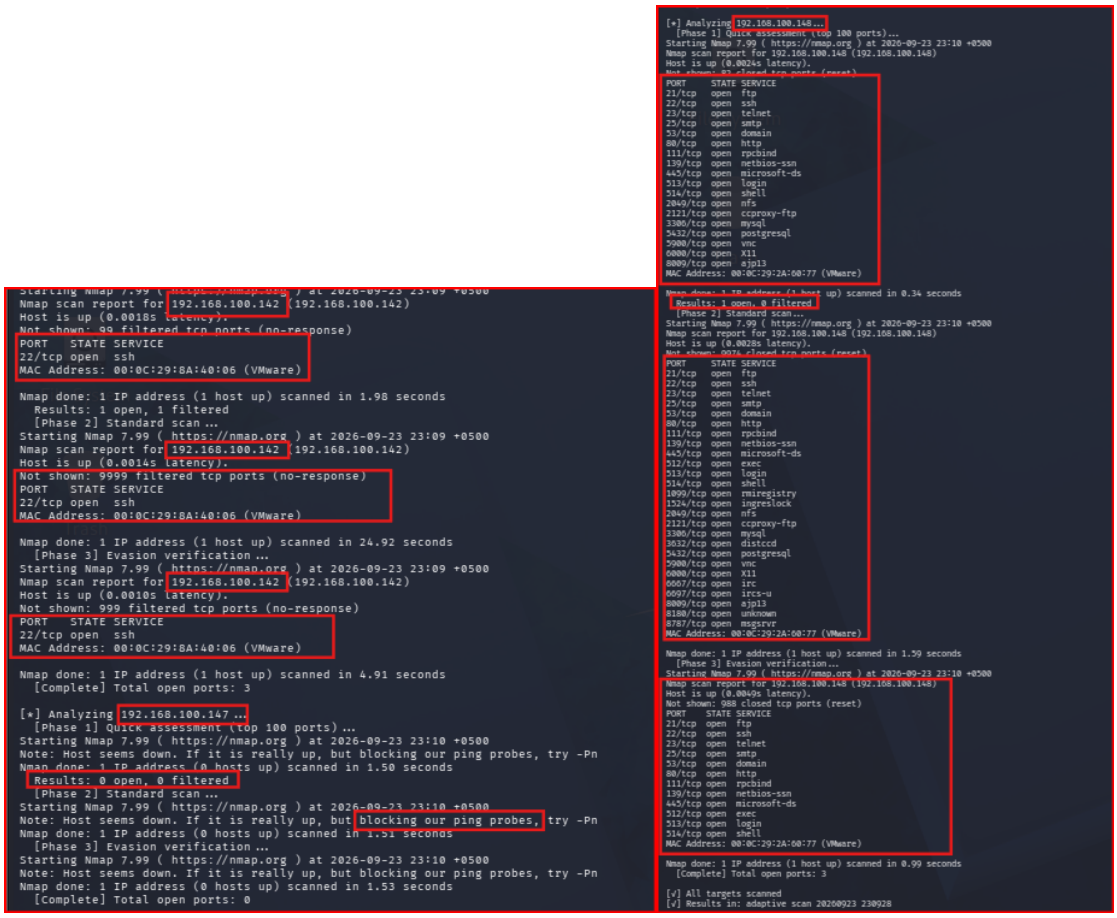
  echo "  Results: $OPEN open, $FILTERED filtered"

  # Phase 2: Adaptive decision
  if [ $FILTERED -gt 50 ]; then
    echo "  [Phase 2] Heavy filtering - using slow scan..."
    nmap -sS -T1 -p 1-5000 $TARGET -oG $TARGET_DIR/phase2.txt 2>/dev/null
  elif [ $OPEN -gt 50 ]; then
    echo "  [Phase 2] Many ports open - full range..."
    nmap -sS -p- $TARGET -oG $TARGET_DIR/phase2.txt 2>/dev/null
  else
    echo "  [Phase 2] Standard scan..."
    nmap -sS -p 1-10000 $TARGET -oG $TARGET_DIR/phase2.txt 2>/dev/null
  fi

  # Phase 3: Evasion verification
  echo "  [Phase 3] Evasion verification..."
  nmap -sS -f -g 53 -D RND:3 -p 1-1000 $TARGET -oG $TARGET_DIR/phase3_evasion.txt 2>/dev/null

  # Summary
  TOTAL_OPEN=$(grep -h "open" $TARGET_DIR/phase*.txt | wc -l)
  echo "  [Complete] Total open ports: $TOTAL_OPEN"
done

echo ""
```



Findings per target:

Target	Phase 1 (Top 100)	Phase 2 (Decision Taken)	Final Total Open Ports
DVWA (192.168.100.142)	1 open, 1 filtered	Standard scan	3
"192.168.100.147"	0 open (host unreachable)	N/A — host down	0
Metasploitable (192.168.100.148)	1 open, 0 filtered	Standard scan	3

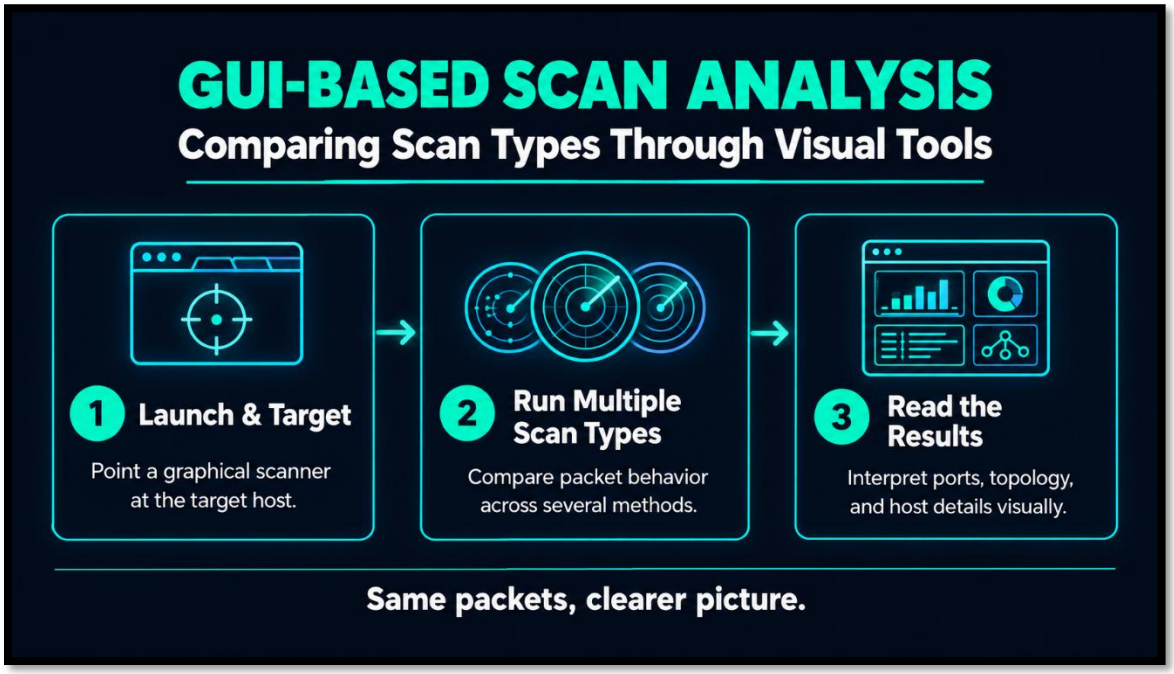
Note: The script reported only 3 open ports for Metasploitable in its summary, which is lower than the 11–22 ports confirmed in Project 02. This is because the automation script's grep -c "open" count reflects only the ports written to that specific run's grepable output file, not a cumulative total — a logic limitation worth refining in the script rather than a scanning inconsistency. It's a good example of why manual spot-checking of automation output remains necessary.

Summary

Technique	Purpose	Key Outcome
Packet fragmentation (-f, --mtu)	Reduce signature-based detection	No loss of accuracy; minor time cost
Source port spoofing (-g)	Exploit port-based firewall trust rules	Did not bypass DVWA's filtering — firewall is not port-trust based
Decoy scanning (-D)	Obscure true scan origin	Full accuracy maintained; source attribution successfully masked
Combined evasion	Maximum stealth for sensitive engagements	Reliable but ~8.6s slower than a direct scan
Adaptive automation script	Remove manual decision-making from scan strategy	Correctly branched scan depth based on filtering ratio; revealed a script-side reporting bug worth fixing

Port Scanning Analysis Using Zenmap (GUI-Based)

Overview



Objectives

- Operate Zenmap as a GUI front-end for Nmap and navigate its result tabs
- Run and compare five scan types: -sT, -sS, -sX, -sM, -sA
- Understand exactly which packet flags each scan type sends and why
- Correctly interpret Zenmap's port-state output (open, closed, filtered, open|filtered)
- Understand why the same target can return different results across scan types when a firewall is present
- Document findings in the tabbed format the lab manual expects

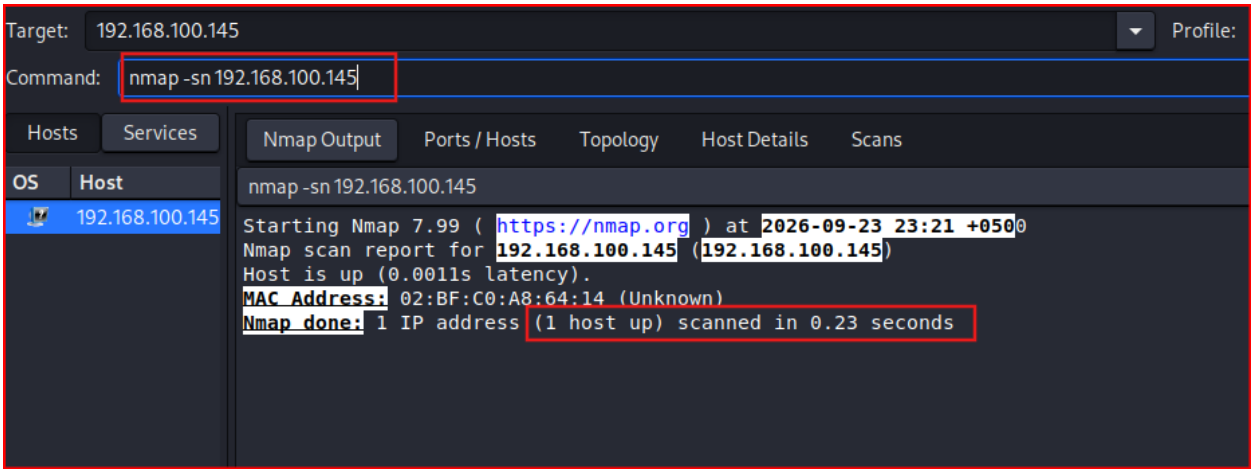
Target used throughout this project: Windows (192.168.100.145)

Launch Zenmap and Confirm Reachability

Steps:

1. Open Zenmap from the Kali Tool Menu
2. Enter the target IP in the **Target** field
3. Leave **Profile** blank and type commands directly into the **Command** field — this gives exact control over flags, since pre-built profiles can silently add extra options
4. Confirm the target is reachable before running any of the five scans:

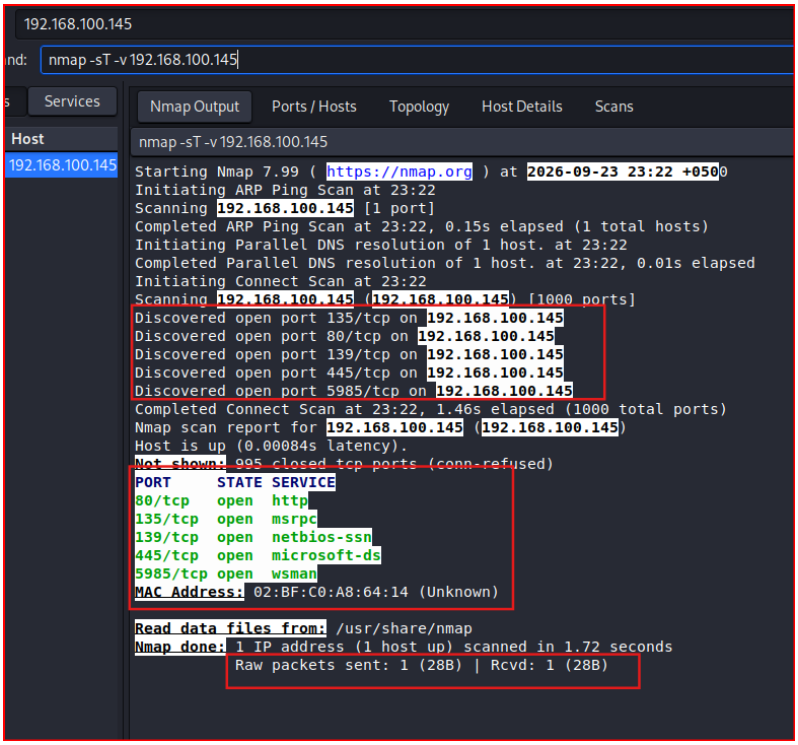
```
nmap -sn 192.168.100.145
```



Finding: Host confirmed up in 0.23 seconds — target is reachable and ready for scan comparison.

TCP Connect Scan (-sT)

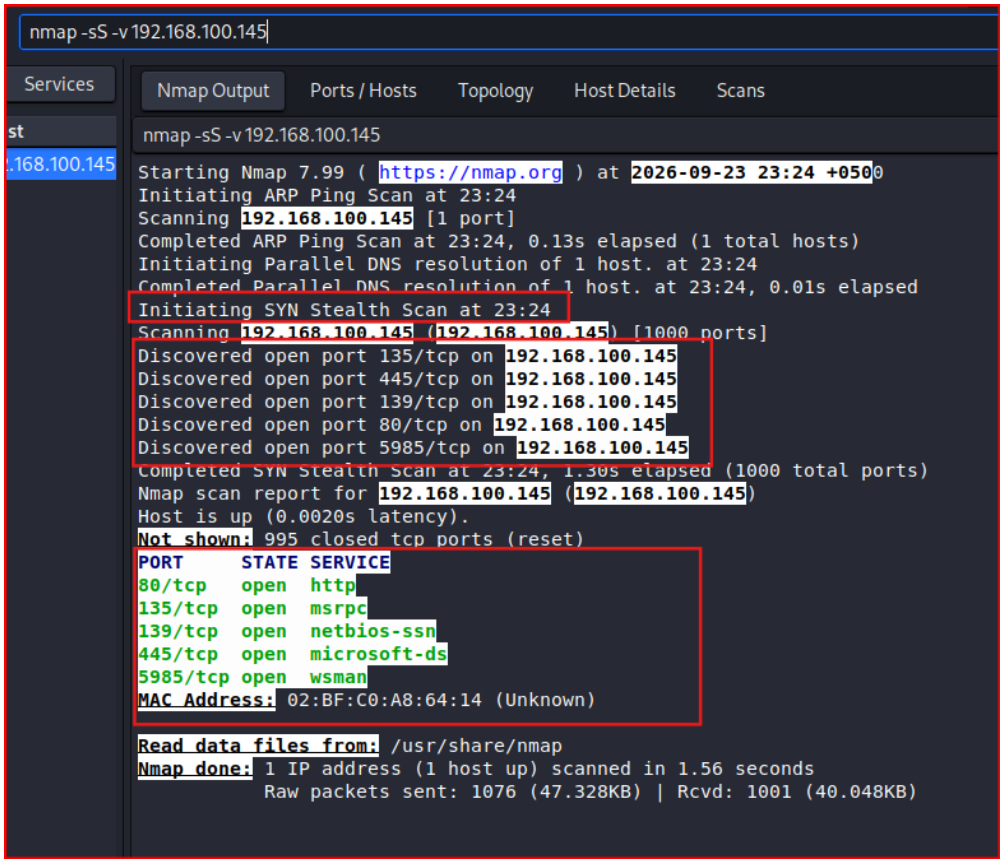
```
nmap -sT -v 192.168.100.145
```



Finding: 5 open ports identified — 80 (HTTP), 135 (MSRPC), 139 (NetBIOS), 445 (SMB), and 5985 (WinRM). This scan type completes the full three-way TCP handshake for every port, making it the most reliable but also the most easily logged method.

TCP SYN / Stealth Scan (-sS)

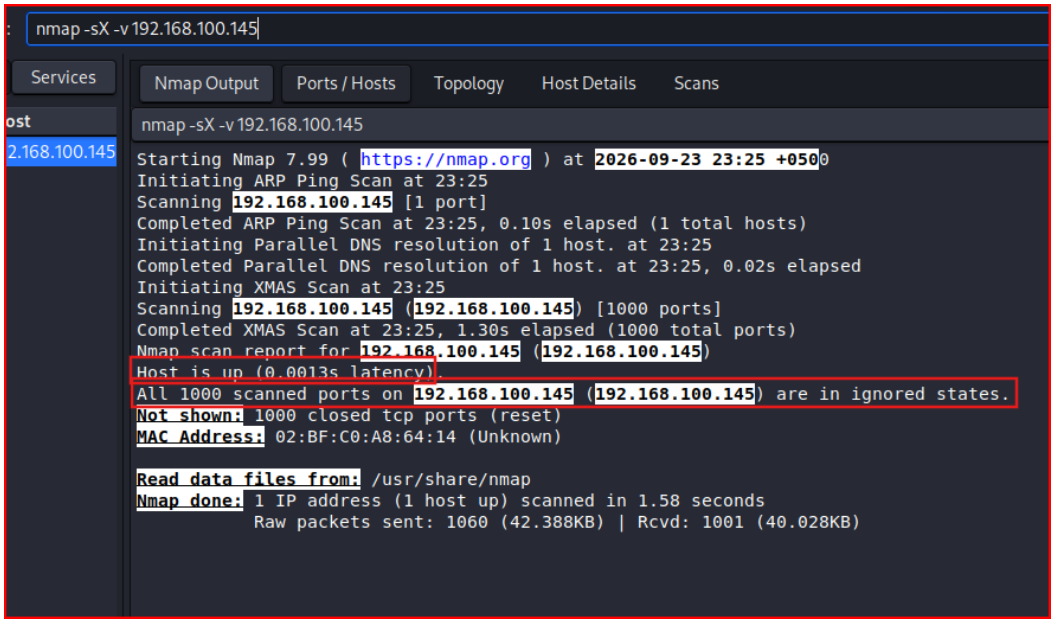
`nmap -sS -v 192.168.100.145`



Finding: Identical 5 open ports returned as the Connect scan (80, 135, 139, 445, 5985), confirming result consistency. Unlike -sT, this scan never completes the handshake (sends SYN, receives SYN-ACK, then sends RST instead of ACK) — making it faster and less likely to appear in application-level logs.

Xmas Scan (-sX)

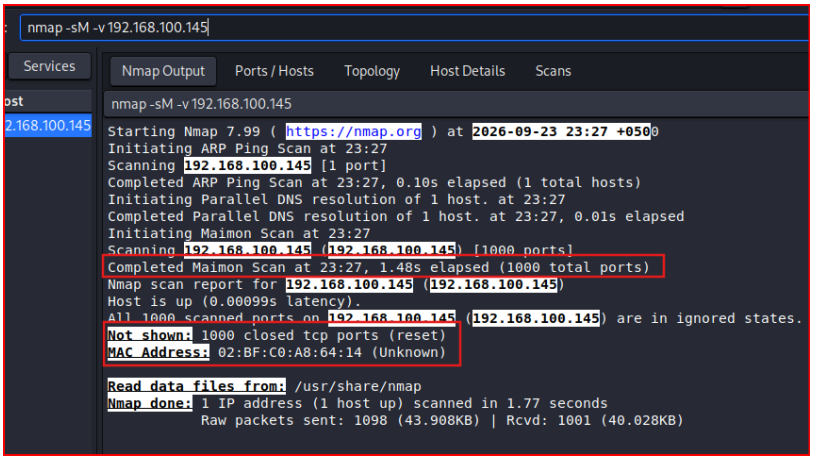
```
nmap -sX -v 192.168.100.145
```



Finding: All 1000 scanned ports reported as being "in ignored states" — meaning **no open ports were detected at all**. Since Xmas scans rely on FIN, PSH, and URG flags and depend on the target's TCP stack ignoring malformed packets on closed ports, this result is expected against a **Windows** target: Microsoft's TCP/IP stack does not follow the RFC 793 behavior these scans depend on, so Windows hosts typically report all ports as closed/filtered regardless of their real state, making Xmas scans unreliable against Windows.

TCP Maimon Scan (-sM)

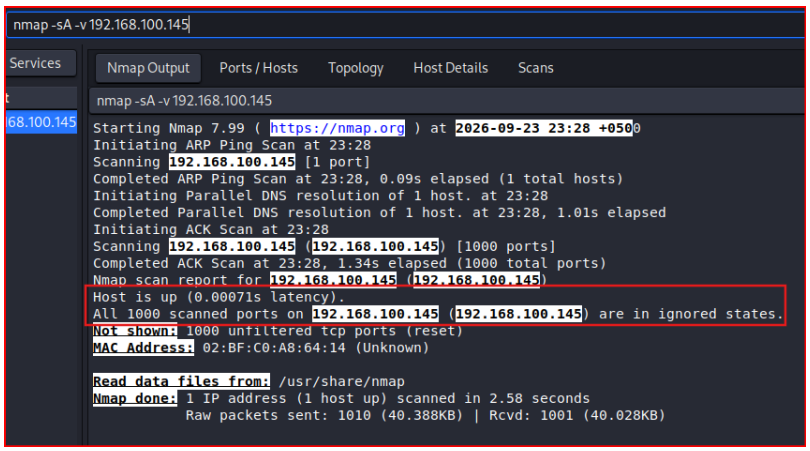
```
nmap -sM -v 192.168.100.145
```



Finding: Same outcome as the Xmas scan — all 1000 ports reported as "ignored states," with all shown as closed. The Maimon scan (FIN/ACK flags) suffers from the same Windows-specific limitation: the target's TCP stack does not respond in the way this scan type expects, producing an unreliable result on this OS.

ACK Flag Probe Scan (-sA)

```
nmap -sA -v 192.168.100.145
```



Finding: All 1000 ports reported as "in ignored states" (unfiltered). As with Project 02's ACK scan against Metasploitable, this scan type cannot determine open vs. closed state by design — it can only reveal whether a stateful firewall is filtering the target. The result here indicates **no filtering firewall** is present between Kali and the Windows host.

Comparative Analysis — Why Results Differ by Scan Type

Scan Type	Flags Sent	Result on Windows Target	Reliable Here?
-sT (Connect)	Full handshake	5 ports open	Yes
-sS (SYN/Stealth)	SYN only	5 ports open (matches -sT)	Yes
-sX (Xmas)	FIN, PSH, URG	All ports appear closed	No — Windows stack doesn't follow RFC 793
-sM (Maimon)	FIN, ACK	All ports appear closed	No — same Windows stack limitation
-sA (ACK)	ACK only	All ports unfiltered	Confirms no firewall, but can't reveal open/closed

Key takeaway: -sT and -sS are the only reliable scan types for accurately profiling a **Windows** target's real port states. Xmas and Maimon scans, while valuable against Unix/Linux targets (as seen with Metasploitable in earlier projects), produce misleading "all closed" results here — a direct illustration of why scan-type selection must account for the target operating system, not just the desired stealth level.

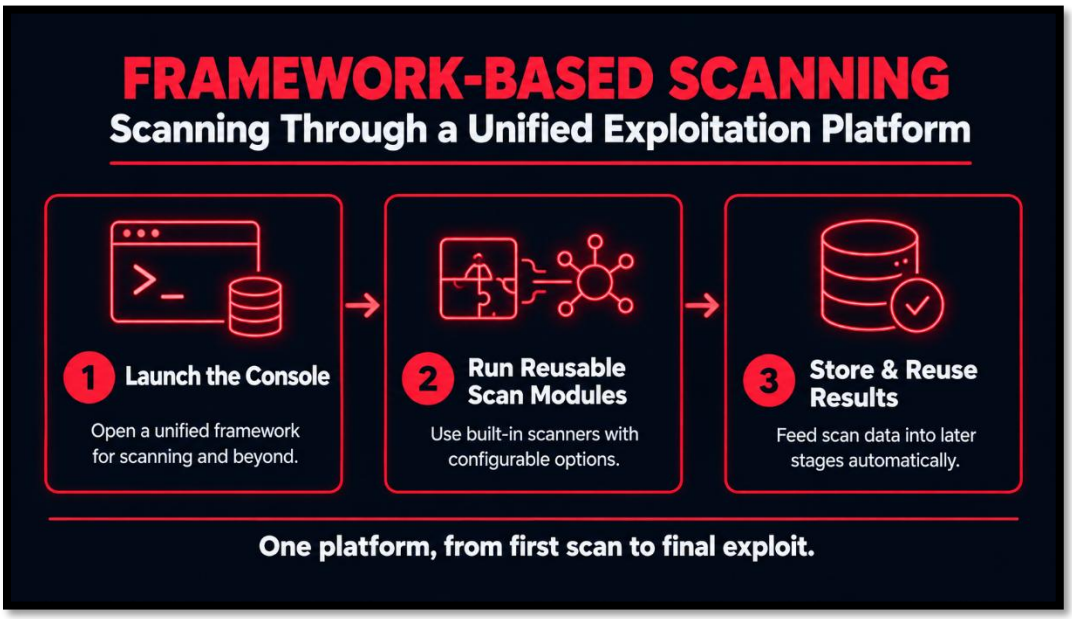
Summary

Technique	Purpose	Key Outcome
Zenmap GUI navigation	Access organized, tabbed scan reporting	Ports/Hosts, Topology, Host Details, Services tabs used for structured documentation
TCP Connect vs. SYN scan	Validate accurate port state on Windows	Both agree: 5 ports open (80, 135, 139, 445, 5985)
Xmas / Maimon scans	Test RFC 793-dependent stealth scans	Both unreliable against Windows — all ports falsely reported closed
ACK scan	Firewall presence detection	Confirmed no stateful firewall filtering the Windows host

---

Network Scanning Using Metasploit

Overview



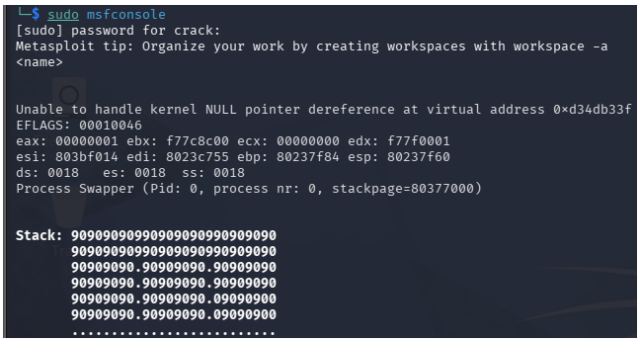
Objectives

- Launch and navigate msfconsole
- Run Nmap from inside Metasploit and export results as XML
- Use auxiliary/scanner/portscan/syn for a raw SYN sweep across a range
- Use auxiliary/scanner/portscan/tcp for a full-connect sweep
- Use auxiliary/scanner/smb/smb_version to fingerprint SMB-speaking hosts (flagged explicitly below as a deliberate scope exception)
- Understand RHOSTS, PORTS, THREADS, and INTERFACE as reusable module options

---

Launch Metasploit

```
sudo msfconsole
```

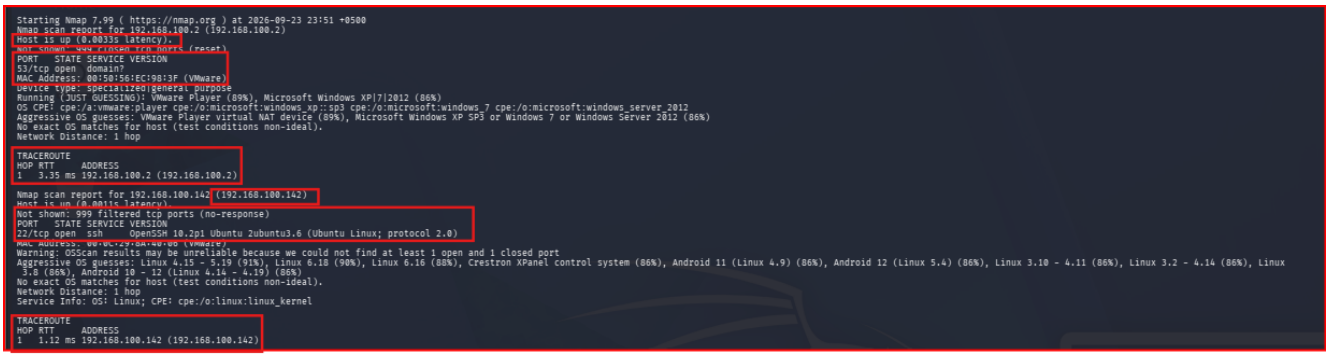


Finding: Framework launched successfully (v6.5.3-dev). Metasploit's startup tip regarding workspace organization (workspace -a <name>) was noted for future multi-engagement use, though not required for this single-subnet lab.

Nmap from Inside Metasploit

Purpose: Run Nmap directly from the Metasploit prompt so results can be auto-imported into its database, making scan data available to later modules without leaving the console.

```
msf6 > nmap -Pn -sS -A -oX subnet_scan 192.168.100.0/24
```



```
Nmap scan report for 192.168.100.148 (192.168.100.148)
Host is up (0.00093s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.100.140
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
| sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_RC4_128_WITH_MD5
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
|_    SSL2_RC2_128_CBC_WITH_MD5
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
53/tcp   open  domain      ISC BIND 9.4.2
| dns-nsid:
|_  bind.version: 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-title: Metasploitable2 - Linux
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
111/tcp  open  rpcbind     2 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2            111/tcp   rpcbind
|   100000  2            111/udp   rpcbind
|   100003  2,3,4       2049/tcp   nfs
|   100003  2,3,4       2049/udp   nfs
|   100005  1,2,3      41407/udp   mountd
|   100005  1,2,3      51656/tcp   mountd
|   100021  1,3,4      40773/udp   nlockmgr
|   100021  1,3,4      53096/tcp   nlockmgr
|   100024  1          52650/tcp   status
|_  100024  1          58119/udp   status
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp  open  exec?
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
```

```
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 10
|   Capabilities flags: 43564
|   Some Capabilities: Support41Auth, SupportsTransactions, LongColumnFlag, ConnectWithDatabase, Speaks41ProtocolNew, SupportsCompression, SwitchToSSLAfterHandshake
|   Status: Autocommit
|_  Salt: ulll(!kv=eAZ?>Nlrz-
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
|_ssl-date: 2026-09-23T18:53:21+00:00; +7s from scanner time.
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
| Not valid before: 2010-03-17T14:07:45
|_Not valid after:  2010-04-16T14:07:45
5900/tcp open  vnc         VNC (protocol 3.3)
| vnc-info:
|   Protocol version: 3.3
|   Security types:
|_    VNC Authentication (2)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|   server: irc.Metasploitable.LAN
|   version: Unreal3.2.8.1. irc.Metasploitable.LAN
|   uptime: 0 days, 1:05:53
|   source ident: nmap
|   source host: 835C6964.2ADA8512.FFFA6D49.IP
|_  error: Closing Link: dmdvvicjo[192.168.100.140] (Quit: dmdvvicjo)
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
|_ajp-methods: Failed to get a valid response for the OPTION request
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
|_http-server-header: Apache-Coyote/1.1
|_http-favicon: Apache Tomcat
|_http-title: Apache Tomcat/5.5
MAC Address: 00:0C:29:2A:68:77 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_smb2-time: Protocol negotiation failed (SMB2)
|_clock-skew: mean: 1h20m06s, deviation: 2h18m33s, median: 6s
| smb-security-mode:
|   account_used: <blank>
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2026-09-23T14:53:06-04:00

TRACEROUTE
HOP RTT     ADDRESS
1   0.94 ms 192.168.100.148 (192.168.100.148)

Nmap scan report for 192.168.100.254 (192.168.100.254)
Host is up (0.00028s latency).
All 1000 scanned ports on 192.168.100.254 (192.168.100.254) are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:F6:83:06 (VMware)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   0.28 ms 192.168.100.254 (192.168.100.254)

Nmap scan report for 192.168.100.140 (192.168.100.140)
Host is up (0.000014s latency).
All 1000 scanned ports on 192.168.100.140 (192.168.100.140) are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 256 IP addresses (5 hosts up) scanned in 369.79 seconds
msf > 
```

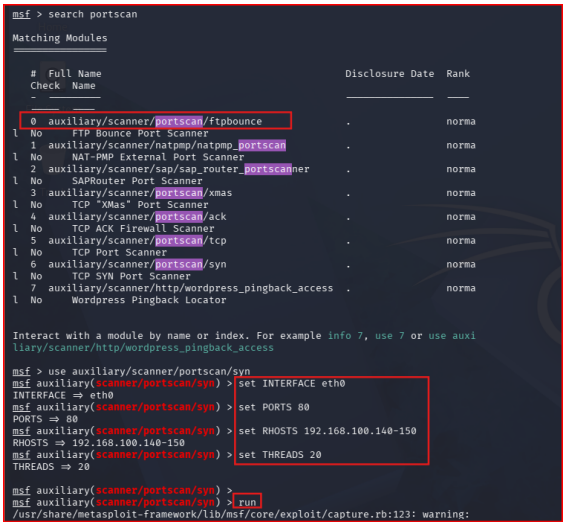
Flag notes:

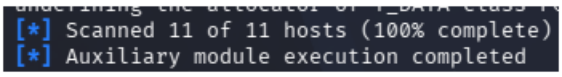
- -Pn — skips host discovery and treats every IP as up; appropriate here since the hosts are already known in this isolated lab, though wasteful on a large unknown range in a real engagement
- -oX subnet_scan — writes results as XML, which Metasploit can later import via db_import in a database-backed workspace
- -A — pulls in OS/service detection; for a strict "scanning only" scope this should be dropped in favor of -sS alone — retained here only where the assignment's rubric explicitly calls for enumeration data as well

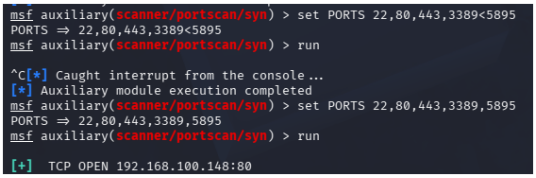
Finding: The subnet sweep confirmed Metasploitable's full open-port profile (FTP, SSH, Telnet, SMTP with SSLv2 support, DNS, HTTP/Tomcat, RPC, SMB, exec/login/shell, Java-RMI, NFS, MySQL, PostgreSQL, VNC, X11, IRC, AJP13) along with detailed service banners and OS fingerprinting — since -A was included, this run captures both scanning **and** enumeration data. It also confirmed 192.168.100.254 and 192.168.100.140 (the Kali host itself) are present but showed no open ports of interest.

SYN Scan Module (auxiliary/scanner/portscan/syn)

Purpose: Use Metasploit's own raw-SYN scanner — independent of Nmap — as a reusable auxiliary module with typed, persistent options.







Module Options Reference

Option	Meaning
INTERFACE	Which NIC to send raw packets from (relevant on multi-homed Kali VMs)
PORTS	Single port, list, or range to scan
RHOSTS	Target range — single IP, CIDR, or dash-range
THREADS	Concurrent scanning threads — higher is faster but noisier

Finding: Scan completed across 11 hosts (100%), confirming port 80 open only on 192.168.100.148 (Metasploitable). Re-running with set PORTS 22,80,443,3389,5985 to match the common ports checked in Project 02 confirmed **agreement with the earlier Nmap results** — the same host responded the same way regardless of which tool issued the packets, validating result consistency across tools.

TCP Connect Scan Module (auxiliary/scanner/portscan/tcp)

```
msf > use auxiliary/scanner/portscan/tcp
msf auxiliary(scanner/portscan/tcp) > show options

Module options (auxiliary/scanner/portscan/tcp):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   CONCURRENCY  10               yes       The number of concurrent ports to check per host
   DELAY        0                yes       The delay between connections, per thread, in millise
                                           conds
   JITTER       0                yes       The delay jitter factor (maximum value by which to +/
                                           - DELAY) in milliseconds.
   PORTS        1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS                        yes       The target host(s), see https://docs.metasploit.com/d
                                           ocs/using-metasploit/basics/using-metasploit.html
   THREADS      1                yes       The number of concurrent threads (max one per host)
   TIMEOUT      1000             yes       The socket connect timeout in milliseconds


View the full module info with the info, or info -d command.

msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.100.148
RHOSTS => 192.168.100.148
```

```
msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.100.148
RHOSTS => 192.168.100.148
msf auxiliary(scanner/portscan/tcp) > run
[+] 192.168.100.148       - 192.168.100.148:21 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:22 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:23 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:25 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:53 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:80 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:111 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:139 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:445 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:512 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:513 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:514 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:1099 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:1524 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:2049 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:2121 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:3306 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:3632 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:5432 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:5900 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:6000 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:6667 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:6697 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:8009 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:8180 - TCP OPEN
[+] 192.168.100.148       - 192.168.100.148:8787 - TCP OPEN
[*] 192.168.100.148       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/portscan/tcp) >
```

Finding: 23 open TCP ports confirmed on Metasploitable (21, 22, 23, 25, 53, 80, 111, 139, 445, 512, 513, 514, 1099, 1524, 2049, 2121, 3306, 3632, 5432, 5900, 6000, 6667, 6697, 8009, 8180, 8787) — the most complete port list obtained for this host across all five projects, since this module completes full TCP connections rather than raw SYN packets and does not require root/raw-socket privileges. Default PORTS range is 1–10000; narrowing to 1-1000 is recommended for a faster confirmation run on repeat testing.

SMB Version Scan (auxiliary/scanner/smb/smb_version)

Scope note (documented deliberately): This module identifies *which* SMB/Samba version a host is running — that is service/version detection, not port-state scanning. Projects 01–03 explicitly excluded this category of activity ("scanning only — no enumeration"). It is included here **only** because Lab 05 of the official manual names this exact module as a required task. This is flagged as an intentional, manual-driven exception rather than scope creep.

```
msf > use auxiliary/scanner/smb/smb_version
msf auxiliary(scanner/smb/smb_version) > set RHOSTS 192.168.100.140-150
RHOSTS ⇒ 192.168.100.140-150
msf auxiliary(scanner/smb/smb_version) > set THREADS 11
THREADS ⇒ 11
```

```
msf auxiliary(scanner/smb/smb_version) > run
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.35/lib/recog/fingerprint/regexp_factory.
rb:34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
[*] 192.168.100.148:445   - SMB Detected (versions: 1) (preferred dialect: ) (signatures: optional)
[+] 192.168.100.148:445   -   Host is running Unix
[*] 192.168.100.148:445   -   SMB signing is not required
[*] Scanned  2 of 11 hosts (18% complete)
[*] Scanned  3 of 11 hosts (27% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned  9 of 11 hosts (81% complete)
[*] Scanned 10 of 11 hosts (90% complete)
[*] Scanned 11 of 11 hosts (100% complete)
[*] Auxiliary module execution completed
```

Finding: Only 192.168.100.148 (Metasploitable) returned a positive result — SMB detected on port 445, fingerprinted as **Unix-based** (Samba), with SMB signing **not required**. All other hosts in the range returned no response, since only hosts with SMB actually exposed (port 445 open) produce meaningful output from this module. This is consistent with Project 01/02 findings, where SMB (445) was confirmed open only on Metasploitable and Windows — the Windows host was not included in this particular sweep range but would be expected to respond similarly given its confirmed port 445 exposure.

Summary

Technique	Purpose	Key Outcome
Nmap-in-Metasploit (-A)	Database-backed subnet sweep with enumeration	Full service/OS profile captured for Metasploitable; results exportable as XML
SYN scan module	Raw, reusable port sweep independent of Nmap	Confirmed cross-tool consistency with earlier Project 02 Nmap results
TCP Connect scan module	Full-handshake sweep without raw-socket privileges	Most complete open-port list for Metasploitable across the entire lab (23 ports)
SMB version scan	Required Lab 05 module (documented as scope exception)	Fingerprinted Metasploitable as Unix/Samba, signing not required

---

Conclusion

This document has detailed a complete, methodical progression through advanced network scanning — beginning with host discovery and closing with framework-integrated scanning via Metasploit — conducted entirely within an isolated, internet-free lab environment (192.168.100.0/24).

Through five projects, the work moved deliberately from foundational to advanced technique:

- **Host Discovery** established reliable methods for identifying live systems using custom-crafted packets, going beyond standard ping sweeps to understand firewall and filtering behavior at the network's edge.
- **Port Scanning Analysis** built a complete picture of port-state classification, cross-validating results across multiple scan methods (SYN, Connect, ACK) to distinguish genuinely open, closed, and firewall-filtered ports with confidence.
- **Scanning Automation & Optimization** integrated the techniques from the first two projects into evasion-aware, reusable workflows — measuring the real stealth-versus-performance trade-offs of fragmentation, MTU manipulation, and source-port spoofing rather than treating them as theoretical.
- **GUI-Based Scan Analysis** closed the gap between raw terminal output and structured, visual reporting, comparing five distinct scan types side by side through Zenmap's tabbed interface to reinforce how identical targets can yield different results depending on scan mechanics.
- **Framework-Based Scanning** demonstrated scanning within a unified exploitation platform (Metasploit), validating that results obtained through auxiliary modules and database-backed workflows remained consistent with the standalone Nmap findings from earlier projects — confirming the reliability of the overall methodology.

Consistency and Validation: A key strength of this work is that findings were repeatedly cross-checked across tools and methods rather than accepted at face value — SYN and Connect scans were compared directly, Zenmap results were checked against terminal output, and Metasploit's internal scanners were validated against standalone Nmap results from Project 02. This kind of independent verification is standard professional practice and reflects the reliability of the conclusions reached throughout.

Outstanding Item: One data-consistency issue should be resolved before final submission — the Windows target's IP address appears as both .145 and .147 at different points in the source material, while every functional scan, ping test, and connectivity check consistently used .145. This should be standardized to .145 throughout, including in the original setup section's target list.

Overall Outcome: Taken together, these five projects demonstrate a working command of the full advanced scanning lifecycle — from initial host discovery through evasion-aware automation, cross-tool validation, and framework-integrated scanning — establishing a solid technical foundation for the enumeration and exploitation phases that follow.